

Стъпка 6 Обобщение — Архитектури за изкуствен интелект в игрите от край до край

Изследване върху възможностите за прилагане на изкуствен интелект в компютърните игри

Alexander Andreev

June 2026

Съдържание

6	Стъпка 6 — Архитектури на игрови изкуствен интелект от край до край	1
6.1	Въведение	1
6.2	DeepStack (2017)	3
6.3	Либратус (2017/2018)	9
6.4	Pluribus (2019)	17
6.5	ReBeL (2020)	24
6.6	Student of Games / SoG (2023)	33
6.7	Синтез	42

6 Стъпка 6 — Архитектури на игрови изкуствен интелект от край до край

6.1 Въведение

Глава 6 е крайъгълният камък на този изследователски план. Стъпки 1–5 сглобяват отделните елементи — игровитеоретичния речник на игрите в разширена форма и равновесията на Наш, минимизирането на контрафактичното съжаление (CFR) и неговите **Monte-Carlo variants**, абстракцията на играта и невронните апроксиматори на функции, които заменят табличното съхранение, когато една игра го надрасне — и тази глава е мястото, където тези елементи се сглобяват в пълноценни системи от състезателно ниво. Подходът е умишлено **архитектурен**: вместо да се извеждат отново математическите зависимости, които са представени в техническите глави и цитираните статии, всяка система се разглежда на нивото на това как е *изградена* — какво изчислява **извън линия**, какво изчислява в **реално време**, къде е обучението и какво всъщност може да гарантира. Кратък отчет за всяка система открива всяка секция, така че същите девет измерения да се подредят с един поглед, а текстът след това проследява дизайнерските решения, инженерните компромиси и изоставените подходи зад тях.

Петте системи обхващат седем години и, прочетени в ред, проследяват еволюцията на свръхчовешкия игрови изкуствен интелект: **DeepStack** (2017), **Libratus** (2017/2018), **Pluribus** (2019), **ReBeL** (2020) и **Student of Games** (2023). Изкушаващо е да се прочете такъв списък като класация — всяка следваща по-силна от предишната — но това нито се е случило, нито така е организирана тази глава. Прогресията е последователност

от *умишлени компромиси*: всяка система придобива нова способност, като се отказва от нещо друго, а най-показателните моменти са тези, в които областта прави странична стъпка или дори отстъпление. **DeepStack** и **Libratus** са почти съвременници, които предлагат *противоположни решения* на едно и също заболяване — **DeepStack** изоставя парадигмата „абстракция и план“, която доминираше в компютърния покер близо две десетилетия, докато **Libratus** я запазва и излекува единствения ѝ фатален недостатък с решаване на под-игри в реално време, което е доказуемо безопасно. **Pluribus** след това пробива *барьерата на броя играчи*, достигайки свръхчовешка игра с шестима участници, но само като се отказва изцяло от гаранции за безопасност и остава изцяло табличен. **ReBeL** се връща към двама играчи, но прави огромен скок в обобщаването — научавайки стойности на състоянието на убеждението със самообучение в стил **AlphaZero-style Self-Play**, разтваряйки както абстракцията, така и плана, и *възстановявайки* безопасността, от която **Pluribus** се беше отказал. **Student of Games**, като завършек, обединява игра с пълна и непълна информация в един алгоритъм — и плаща за тази широта с пикова сила, губейки решително от специализиран **AlphaZero** в Го. Прочетено по този начин, главата е изследване на цената на всеки напредък, а не класация на победители.

Под отделните компромиси три оси на движение преминават през всичките пет системи и придават гръбнака на главата. Първата е **представителна**: преходът от ръчно изградена *абстракция* — групиране на подобни раздавания и допускане само на няколко размера залози — към *научено невронно приближение*, при което мрежата обобщава ситуации, които таблица никога не би могла да изброи. Втората е **времева**: преходът от *решаване на цялата игра извън линия* в съхранена стратегия към *научаване на компактен модел и търсене с него в реално време*, така че изчислението се извършва в момента на вземане на решение, а не е предварително заложено. Третата ос се появява едва в самия край, със **Student of Games**: обединяването на игра с **пълна и непълна информация** в един надежден алгоритъм, съчетаващ двете големи традиции на игровия изкуствен интелект — линията минимакс / Monte-Carlo-tree-search / AlphaZero и линията CFR / покер — които са се развивали по отделни пътища в продължение на седемдесет години. Първите две оси описват четирите покер системи; третата е това, което прави петата завършек на главата.

Една идея свързва тези нишки и заслужава да бъде назована преди самите системи, именно защото тя *не е* една от тях: **ограничено по дълбочина решаване**. Формализирано от Brown & Sandholm (2018), то е принципът, че може да се търси само малко напред в игра с непълна информация и остатъкът да се замести с *научена или предварително изчислена стойност* — при условие че заместването се извършва така, че скритата информация да не го прави невалидно. Това е теорията, която ретроспективно обединява непрекъснатото пререшаване на **deepStack** с вложеното решаване на под-игри на **Libratus**, която стои в основата на продължителните стратегии на **Pluribus** и която, във формата на състояние на убеждението, се превръща във вътрешния цикъл на **Recursive Belief-based Learning** и **Student of Games**. Ние го разглеждаме като свързваща тъкан — споменавано навсякъде, където една система го реализира — а не като шеста точка, защото ограниченото по дълбочина решаване е механизмът *чрез който* оста извън линия към търсене всъщност функционира.

Накрая, тъй като тази глава е връзката между основите на стъпки 1–5 и изследванията върху моделирането на противника и експлоатацията в стъпки 7–15, си струва още в началото да се посочи единствената нишка, към която синтезът се връща. Всяка система тук е, по умишлен дизайн, **независима от противника**: всяка изчислява стратегия, която е трудна за побеждаване *в най-лош случай* и след това я прилага без да обръ-

ща внимание кой всъщност седи срещу нея — Pluribus дори не знае самоличността на своите опоненти, а както тя, така и Libratus отказват по принцип да ги моделират или да се адаптират към тях, за да не бъдат контраексплоатирани в отговор. Този подход, поставящ устойчивостта на първо място, е голямата сила на областта и, за дисертацията относно *адаптивна* игра, нейното определящо ограничение: именно съзнанието за противник, което тези системи пропускат, стъпки 7–15 се стремят да добавят. Затова главата завършва не с победител, а със синтез — карта на това, което петте системи споделят, какво всяка от тях е жертвала и къде всъщност се намират откритите проблеми, които мотивират останалата част от тази работа.

6.2 DeepStack (2017)

DeepStack (Moravčík et al., 2017), от компютърно-покер групата на Университета на Алберта с колаборатори в Прага, беше първата програма, която побеждава професионални покер играчи в хедс-ъп безлимит покер (HUNL) със статистическа значимост и първата, която поставя евристичното търсене — двигателят зад шахмата и Го — на теоретична основа в игра с непълна информация. В рамките на четириседмично проучване тя победи група от 33 професионалисти с 492 мили-големи блайнда на игра (mbb/g, стандартната покер единица за процент победи; 50 mbb/g представлява значително професионално предимство) в 44 852 ръце и нито една известна техника не успя да открие пропуск в играта ѝ. Нейният водещ *какво-ако* въпрос е този, който рамкира цялата тази глава: **може ли програма да играе покер по начина, по който alphaGo играе Го — търсейки локално от текущата ситуация и доверявайки се на научена функция на стойността за всичко извън хоризонта — дори когато в покера самата ситуация е частично скрита?**

С един поглед	DeepStack (2017)
Играчи	2 (хедс-ъп)
Тип игра	HUNL — хедс-ъп безлимит тексаски но-лимит покер за двама (двуйграчева игра с нулева сума)
План (офлайн)?	Не — офлайн обучението е за мрежите за стойност, а не за съхранена стратегия
Невронна компонента	Deep Counterfactual Value Networks (флоп, търн, спомагателни); само стойност
Механизъм за търсене	Продължително пре-решаване (ограничено по дълбочина предвиждане с CFR при всяко решение)
Абстракция?	Няма ограничения в играта; 1000-кълъстерна кълъстеризация само на входа на мрежата, плюс оскъдни залози и групиране на действията на ривъра при предвиждането
Също и с пълна информация?	Не (само непълна информация)
Изчислителна мощ	Офлайн-интензивна (~175 процесорно-ядрени години за етикетирание на мрежата от ходове); времето за игра работи на една GPU, < 5 s/решение

С един поглед	DeepStack (2017)
Ключова иновация	Продължително пре-решаване + научени контрафактични стойности: първото <i>звуково</i> евристично търсене за игри с непълна информация

6.2.1 Затворената празнина

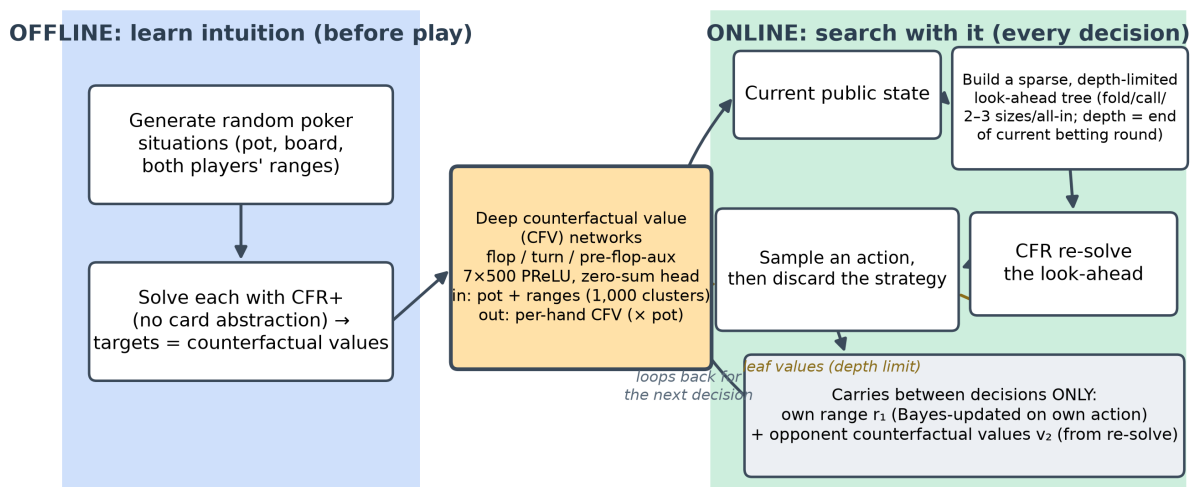
Всеки предишен етап в развитието на изкуствения интелект за игри — табла, шах, Го — се основаваше на *локално търсене*: от текущата позиция се разглеждат няколко хода напред и останалото се замества с евристична стойност. Тази рецепта предполага, че позицията е известна, а покерът нарушава това предположение — правилното действие зависи от вероятностното разпределение на скритите карти на опонента, което се разкрива единствено чрез неговите залози, а те от своя страна зависят рекурсивно от предварителните му убеждения относно вашите карти. Отвореният въпрос, който deepStack се опита да реши, е дали евристичното търсене може да бъде направено *звуково* при наличие на скрита информация в мащаба на HUNL — игра с приблизително 10^{160} решавачи точки — сравнима по обем с Го, но с непълна информация.

Почти две десетилетия доминиращият отговор беше нещо съвсем различно: **абстракция плюс офлайн равновесие плюс превод**. Свийте 10^{160} ситуацияите на HUNL до приблизително 10^{14} абстрактни такива чрез групиране на подобни ръце (*абстракция на карти*) и позволяване само на няколко размера залози (*абстракция на действията*), решете тази по-малка игра офлайн с минимизиране на контрафактичното съжаление (CFR), за да произведете пълна стратегия — *план* — съхранете я и в момента на играта *преведете* всяка реална ситуация и залог на опонента към най-близката абстрактна. Компресията е загубна и загубата се проявява като експлоатируемост — колко може да спечели опонент в най-лошия случай, качественият показател на областта, нула при равновесие на Наш. През 2015 г. абстрактно-базираната програма Claudico загуби от професионалисти с 91 mbb/g, а сондаж за най-добър локален отговор (LBR, лесноизчислима долна граница на експлоатируемостта) по-късно показва, че топ състезателните ботове са експлоатируеми с повече от 3000 mbb/g — четири пъти по-зле, отколкото ако се откажеш от всяка ръка. Плановите също бяха огромни (една стратегия без абстракции на картите отне около 2 TB и 14 процесорно-ядрени години за изчисляване и все още беше *експлоатируема* чрез залози извън дървото), а стъпката на превода сама по себе си беше източник на слабост. DeepStack затваря тази празнина, като изоставя цялата конструкция: никога не изгражда абстракция на цялата игра и никога не съхранява план, разсъждавайки за всяка ситуация *така както тя действително възниква* и замествайки само далечното остатъчно продължение на играта с научена оценка.

6.2.2 Архитектура

deepStack се разделя ясно на **извън линия** фаза за изграждане на интуиция и **онлайн** фаза за търсене с нея (Фигура 6.1).

Извън линия системата генерира милиони случайни покер ситуации и ги решава с cFR решавач, за да получи целеви *контрафактични стойности* — условни „какво-ако“ печалби при държане на всяка възможна **раздаване** — и тези двойки (ситуация → стойност) обучават **дълбоките контрафактични мрежи за стой-**



Фигура 1: DeepStack architecture: offline intuition-building (left) feeding a shared deep counterfactual-value network, reused as the leaf evaluator inside the online continual-re-solving loop (right).

ности (CFV Network). Онлайн deepStack не използва план: между отделните решения той запазва само два вектора — своя *диапазон* (разпределението върху възможните раздавания, които може да държи) и *контрафактичните стойности* на опонента — а при всеки ход изпълнява cFR върху малко дърво за предвиждане, вкоренено в истинското текущо състояние, като използва CFV Network за изчисляване на стойностите на листата на границата на дълбочината. Тъй като cFR вече изчислява както диапазоните, така и контрафактичните стойности, той се вписва по естествен начин в този цикъл.

Това е евристично търсене с три компонента: надеждно изчисляване на локална стратегия (продължително пререшаване), предвиждане с ограничена дълбочина, при което научената функция на стойността замества останалата част от играта, и ограничен набор от действия за предвиждане, който поддържа всяко решаване бързо. Невронната мрежа се използва *само* на границата на дълбочината като оценител на листата; търсенето се извършва от CFR; и — най-важното — в цикъла не се прилага абстракция на цялата игра.

6.2.3 Ключова иновация: продължително пре-решаване с научени контрафактични стойности

Приносът на deepStack е в обединяването на две идеи, като всяка от тях прави другата осъществим.

Първата е **продължително пре-решаване**: възстановяване на свежа локална стратегия при всяко решение въз основа на поддържащия диапазон и вектора на контрафактуални стойности на опонента, след което тя се изхвърля, така че агентът никога не съхранява или не се ангажира с глобална стратегия. Класическото пре-решаване (Burch и др., 2014) показва, че за да се възстанови стратегия за под-игра, не е необходима цялата стратегия — достатъчни са само собственият диапазон при влизане в под-играта и векторът на контрафактуалните стойности на опонента. DeepStack избутва това до предела: то *никога* не съхранява стратегия за цялата игра. Всеки път, когато трябва да действа, то пререшава текущото публично състояние въз основа на тези два вектора, извършва едно действие и изхвърля стратегията.

Това, което прави това възможно, е воденето на отчетност. След всяко събитие deepStack актуализира своите

два вектора по прости правила: при **собствено действие** то заменя пререшените контрафактични стойности за избраното действие и извършва байесови актуализации на своя диапазон; при **случайно събитие** то заменя стойностите на тази карта и анулира вече невъзможните раздавания; а при **действието на опонента** то не прави *абсолютно нищо*. Този последен момент е тихият майсторски ход: тъй като то проследява *стойностите* на опонента, а не неговия *диапазон*, и никога не се нуждае от конкретното действие на опонента, за да поддържа тези стойности, то заобикаля стъпката на преобразуване на действие, която парализира абстракционните ботове. Опонентът може да направи всеки залог с всякакъв размер; deepStack просто пререшава от състоянието, което този залог е произвел.

Втората идея е **научена контрафактична стойност мрежа** — „интуицията“ на DeepStack. В игра с пълна информация оценител на възел съпоставя едно състояние към едно число; при непълна информация той трябва да съпостави *цяло публично състояние плюс диапазоните и на двамата играчи* към *вектор* от контрафактични стойности, по една за всяка ръка, защото стойностите в един възел се променят с диапазоните, които достигат до него. DeepStack научава това съпоставяне с директна мрежа от седем скрити слоя с по 500 единици, приемайки размера на пота и двата диапазона (компресирани в 1000 ръчни клъстера) като вход и извеждайки контрафактични стойности за всяка ръка като части от пота. Специално външно ниво налага ограничението с нулева сума: то формира двете имплицитни стойности на играта от диапазоните и суровите изходи и изважда половината от тяхната сума, така че оценките да са взаимно съгласувани и цялото нещо да остане диференцируемо. С тази мрежа, която доставя стойности в края на текущия рунд за залози, преизчисляването с ограничена дълбочина свива играта от 10^{160} решаващи точки до около 10^7 — достатъчно малко, за да бъде решена за по-малко от пет секунди на една GPU.

Сдвояването е доказуемо коректно. Ако грешката на мрежата за стойности е най-много ϵ , а преизчисляването изпълнява T итерации на CFR, експлоатируемостта на получената стратегия е ограничена от

$$\text{exploitability} < k_1 \epsilon + k_2 / \sqrt{T},$$

с игрово-специфични константи k_1, k_2 . Първият член представлява цената на несъвършената интуиция, а вторият — обичайната **сходимост на CFR**. Тази граница е теоретичното ядро на статията — гаранцията, че евристичното **търсене** може да бъде пренесено в условия на **непълна информация**, без **стратегията** да стане **експлоатируема** — и същият шаблон „**ограничено по дълбочина решаване** плюс **научена стойност**“, формализиран допълнително от Brown & Sandholm през следващата година, е в основата на всяка система в тази глава.

6.2.4 Ограничения, задънени улици и недостатъчно описаното в статията

Чистият разказ по-горе представлява основния текст, докато инженерната реалност се съдържа в приложението, където именно акцентът върху „ограниченията и еволюцията“ оправдава своето място.

Най-важната забележка е, че **внедрената система не е тази, за която важи теоремата**. За да играе с човешка скорост, deepStack ограничава своето предвиждане до ограничен набор от залози (пас, залог, два или три размера на залога, ол-ин) и в статията ясно се посочва, че това „отменя свойството на коректност на Те-

орема 1⁴. Така гаранцията на внедрената система е емпирична — подкрепена от резултатите LBR по-долу — а не доказана. Второ отклонение утежнява това: доказателството за коректност предполага стойности на *най-добър отговор*, но deepStack всъщност използва стойности от *самообучение*, които „нямат теоретично обоснование“, но са били по-малко експлоатируеми в ранните тестове. Доказаният алгоритъм и печелившият алгоритъм, строго погледнато, са различни алгоритми.

Абстракцията също така се промъква в периферията. deepStack рекламира, че не използва абстракция на карти за ограничаване на играта — но *прави* групиране на ръцете в 1000 кофи на входа на мрежата за стойности и на ривъра изцяло се отказва от мрежата, решавайки до края на играта, като използва **абстракция на действия с кофи** за постигане на управляемост. За да ускори пререшаването, тя също така прави **топли стартове** на диапазона на опонента, в консервативна комбинация през повечето време, но в агресивен вариант (когато действа първа), който „жертва гаранциите за пререшаване, когато оценката на диапазона на опонента е грешна“. И префлопът, далеч от евтин, изисква изброяване на всички 22 100 възможни флопове чрез флоп мрежата, смекчено единствено чрез кеширане на повтарящи се последователности от залагания. И накрая, изборът на приспособление за пререшаване (приспособлението CFR-D пред алтернативата с максимален марж) е направен, защото то „се представи по-добре в ранните тестове“, а не е изведено — и, отразявайки Стъпка 5, полезната дълбочина на мрежата беше ограничена от данните, а не от архитектурата, като грешката при валидиране се изравняваше след петия слой при бюджет от десет милиона проби. Нито едно от тези не подкопава резултата, но заедно те показват, че заглавието „коректно търсене без абстракции“ е стремеж, който имплементацията приближава, а не постига.

6.2.5 Изчислителна мощност и достъпност

Разходите на deepStack са почти изцяло **извън линия и почти изцяло в CFR solving**, използвано за **етикиране на мрежите за стойности** — не в обучението на невронна мрежа и не във времето за игра. Генерирането на целите означаваше приблизително решаване на милиони случайни под-игри: самата **мрежа от ходове** изразходва около **175 процесорно-ядрени години** на 6,144-ядрен клъстер, като **флоп мрежата** добави приблизително половин **GPU-година** на 20 GPU; за разлика от това, самите мрежи се обучиха за около два дни всяка на едно GPU и по време на игра deepStack работи на **едно стандартно GPU с под пет секунди на решение**. Формата на тази сметка е истинският урок: „интуицията“ се купува веднъж, предварително, чрез грубо **намиране на равновесие**, след което внедряването е евтино — огледален образ на система, чиито разходи са доминирани от **извод**. По отношение на достъпността това постави *от нулата* изграждането в обсега само на добре ресурсирана лаборатория по онова време (**предварителната цена** е с мащаб на клъстер), въпреки че обучената система след това работеше на хардуер, който всеки ентузиаст притежава; десетилетие по-късно тази **предварителна цена** е много по-постижима на стандартни **облачни изчисления** и с по-бързи модерни решаващи устройства, а публична референтна имплементация на **ледюк** допълнително намалява бариерата за влизане.

6.2.6 Силни страни и ограничения

Централната сила на deepStack е **коректност с ниска експлоатируемост**: това е първият метод за търсене при непълна информация с реална гаранция, като емпирично LBR — което излага състезателните ботове

като губещи хиляди mbb/g — не успява да намери начин да го победи, а самият той печели над 350 mbb/g. Той **не изисква пълна абстракция на играта и преобразуване на действие**, така че залозите на опонента извън дървото се обработват точно, без закръгляне; неговата **времеемкост на играта е скромна**; и той се обучава **без човешки данни и с минимални експертни познания**. Допълнително предимство е оценъчната синергия — функцията на стойността на deepStack съвпада с изискванията на оценяващия метод за намаляване на дисперсията AIVAT, като намалява стандартното отклонение в човешките проучвания с 85% и прави постигането на значимост възможно само в няколко хиляди ръце.

Ограниченията определят дневния ред за останалата част от главата. DeepStack е **само двуигрови игра с нулева сума**; неговата **коректност** се основава на тази структура. Той запазва **рядка абстракция на залозите** при предвиждането и **абстракция на действията** на ривъра, както и наследява **грешка на приближението на мрежата за стойности** (член $k_1 \epsilon$), най-голяма на флопа, където разчита най-много на мрежата. Той плаща **високата предварителна цена**, отбелязана по-горе и **преизчислява от нулата** при всяко решение, като е валидиран единствено срещу хора и LBR — никога в директен сблъсък срещу най-силните абстрактни ботове.

6.2.7 Наследство и съвременна значимост

Отстранете специфичните за покера детайли и основната идея на DeepStack е **търсене с ограничена дълбочина с научен модел на стойност в листата, адаптиран към скрита информация** — аналогът при непълна информация на търсенето с насочване по стойност зад AlphaGo и AlphaZero. Тази идея не остаря и не беше забравена; тя се превърна в шаблон. Brown & Sandholm формализираха **теорията на решаването с ограничена дълбочина** през следващата година, а парадигмата след това беше обобщена в пълни **рамки за обучение с подкрепление и търсене**: ReBeL (2020) преформулира научаването на стойността на листата около публични състояния на убеждения със **AlphaZero-style самообучение**, а Student of Games (2023) — който споделя няколко автори с DeepStack — обедини **игра с пълна и непълна информация** в един алгоритъм. Същата „рецепта за **планиране в цикъл с научен модел на стойност**“ по-късно достигна отвъд **двуигровия покер с нулева сума**, например в човешкото ниво на игра на Diplomacy на CICERO. Още по-широко, DeepStack е ранен и чист пример за принципа, който сега е централен за водещия изкуствен интелект: **изразходвайте изчислителна мощност във времето на вземане на решение чрез търсене, насочвано от научена интуиция, вместо да вграждате всичко в една гигантска предварително изчислена стратегия** — същата теза за „**тест-тайм изчислителна мощност**“ зад днешните **модели с възможност за мислене**, с уточнението на самата област (изказано от Noam Brown), че непълната информация се нуждаеше от *осъзнато убеждение* търсене именно защото обикновеното **търсене в дърво на Монте Карло**, което работи за Го, не работи за покер.

Няколко конкретни подсистеми остават използвани и днес: самият **шаблон „ограничена дълбочина плюс научена стойност на листо“**; **диференцируемата глава за изходна нулева сума** за двуигрови мрежи за стойности; **намаляване на дисперсията в стил AIVAT**, което превръща научен модел на стойност в контролна променлива за нискодисперсна оценка на всеки стохастичен агент; и малкият, но преносим трик да се изразяват стойностите като **части от пота** за обобщаване, инвариантно спрямо мащаба. Това, което е наистина остаряло, е ръчно проектираната техника *около* идеята — клъстеризацията k-средни със 1000 кофи,

отделните мрежи за всеки рунд, ръчно настроеното пререшаване — които по-общите научени представяния на ReBeL и Student of Games заменят. Честната оценка: **като внедрена покер система DeepStack е остаряла, но далеч не е просто стъпало** — нейната централна парадигма спечели и сега е основна; а за тази конкретна дисертация нейното състояние (диапазон, контрафактични стойности на опонент) и границата на разпространение на грешката са директни семена за базирано на убеждения моделиране на опонента (**принос 1**) и анализ на безопасното използване (**принос 2**).

6.3 Либратус (2017/2018)

DeepStack отговори на своето *какво-ако*, като изостави парадигмата „абстракция и план“, но все пак запази рядка абстракция на залозите в собственото си предвиждане и никога не беше тестван в директен сблъсък срещу най-силните предварителни ботове или срещу човешки играчи в дълъг, строг мач. Libratus (Brown & Sandholm, 2017; Carnegie Mellon) — на латински *балансиран*, както в приближаване към равновесие на Наш, и *настойчив*, за своята игра — беше създаден независимо и обявен същата година, и направи противоположния залог: запази парадигмата „абстракция и план“ и излекувай нейната единствена фатална болест. Неговият водещ въпрос е огледален образ на този на DeepStack: **какво ще стане, ако решим груб план на цялата игра извън линия, след това го поправим в реално време там, където абстракцията е твърде груба — с доказуема гаранция, че поправката никога няма да ни направи по-експлоатируеми?** През януари 2017 г. Libratus стана първата програма, която победи най-добрите човешки играчи в дълъг мач, побеждавайки четирима професионалисти с 147 мили-големи блайнда на игра (мили-големи блайнда на игра, единицата за процент победи от предишния раздел; ~50 мили-големи блайнда на игра е значително професионално предимство) в 120 000 ръце при 99,98% значимост — и, за разлика от DeepStack, първо демонтира предишния най-добър изкуствен интелект за покер в директен сблъсък.

С един поглед	Libratus (2017/2018)
Играчи	2 (хедс-ъп)
Тип игра	HUNL — хедс-ъп безлимит покер за двама (игра с нулева сума между двама играчи)
План (извън линия)?	Да — абстрахирана стратегия за цялата игра, решена извън линия с MCCFR (подробна в началото, груба в края)
Невронна компонента	Няма — чисто табличен CFR / абстракция (без невронни мрежи никъде)
Механизъм за търсене	Вложено безопасно решаване на под-игри (решаване в реално време на късната игра с CFR+ и преизпълнение за всеки залог на противника извън дървото)

С един поглед	Libratus (2017/2018)
Абстракция?	Да — абстракция на карти (търн/ривър, само в плана) + асиметрична абстракция на действията; разтворена до <i>без абстракция на карти</i> вътре в под-игрите в реално време
Също и с пълна информация?	Не (само непълна информация)
Изчислителна мощ	Тежка извън линия и онлайн: ~25 милиона часа ядро на процесор на свръхкомпютъра Bridges; ~50 възела и десетки секунди за всяко късно решение; без GPU
Ключова иновация	План + вложено в реално време <i>безопасно</i> решаване на под-игри + самоусъвършенстване: точни, доказуемо безопасни отговори на залози извън дървото вместо преобразуване на действие

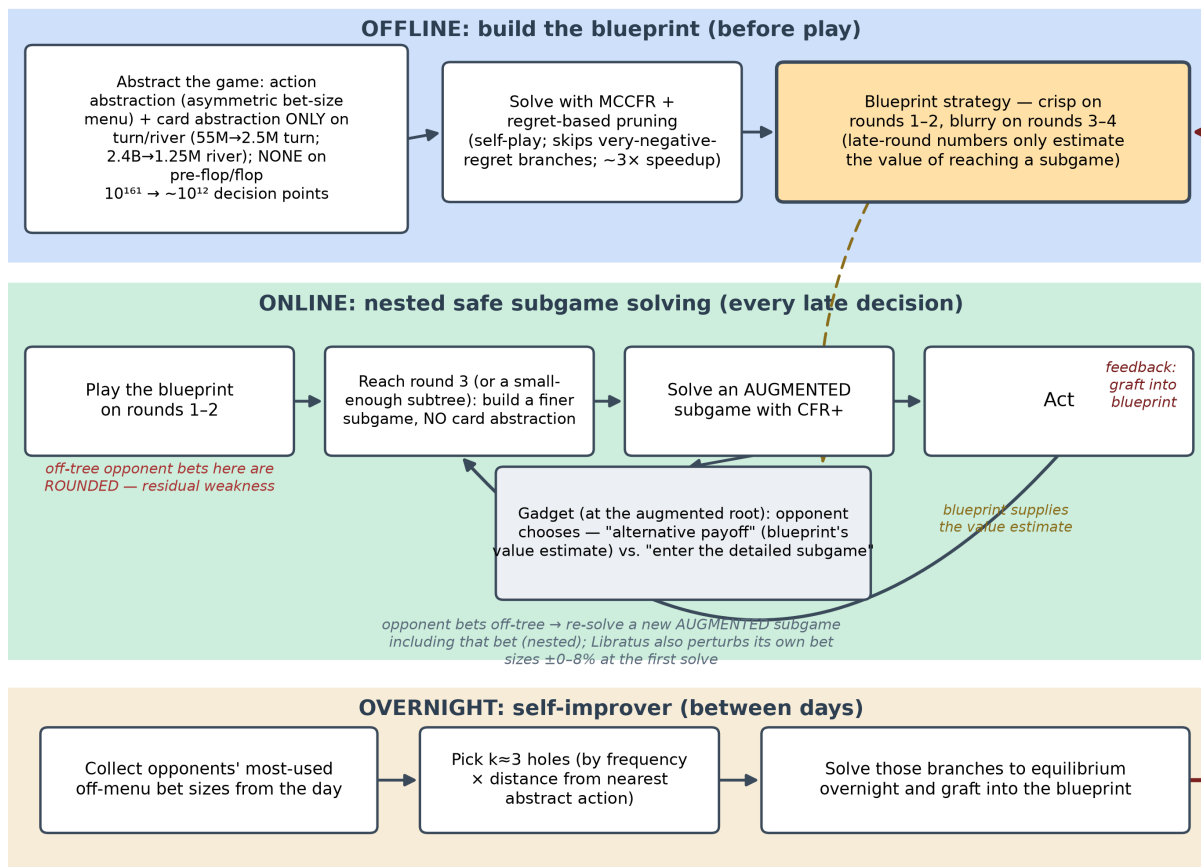
6.3.1 The gap it closed

Предишният раздел очерта парадигмата, която **DeepStack** отхвърли — абстракция плюс офлайн равновесие плюс превод — и посочи нейния фатален недостатък. **Libratus** се насочва точно към този недостатък, без да отхвърля парадигмата. Планът се решава предварително върху *фиксирано* меню от размери на залози; по време на игра всеки залог на противника, който не е в това меню, се *превежда* — закръгля се до най-близкия размер, който ботът познава — след което ботът отговаря така, сякаш противникът е направил закръгления залог. Това закръгляне е единствената най-голяма експлоатируема слабост в абстрактно-базирания покер: сондаж за най-добър локален отговор показва, че водещите състезателни ботове губят хиляди мили-големи блайнда на игра от най-лошия възможен противник, а през 2015 г. предшественикът на **Libratus** — **Claudico** — загуби първия мач *Мозъци срещу III* срещу професионалисти с 91 мили-големи блайнда на игра, отчасти защото опонентите можеха да усетят и накажат неговите граници на превод.

Концептуалният въпрос, на който **Libratus** отговаря, следователно е по-тесен и по-целенасочен в сравнение с този на **deepStack**: *може ли парадигмата на абстракцията, съществуваща от десетилетия, да бъде издигната до свръхчовешко ниво чрез поправка единствено на поведението ѝ в реално време — а именно точното отговаряне на залози извън менюто вместо чрез закръгляне — и може ли тази поправка да се извърши с гаранция, че локалните корекции никога не увеличават глобалната експлоатируемост?* Отговорът е тримодулната архитектура по-долу. Докато **deepStack** премахна напълно плана и разчиташе на научена оценъчна функция, **Libratus** запазва плана като евтин скелет и добавя решавач в реално време, който, когато противникът излезе извън дървото, изгражда и решава по-фина под-игра *съдържаща действителния залог* и вплита резултата обратно в плана. Същата година тогава предложи две противоположни решения на един и същ проблем — едно невронно и без план, друго таблично и базирано на план — и **Libratus** е доказателството, че по-старата парадигма, правилно поправена, все още беше достатъчна, за да достигне свръхчовешка игра.

6.3.2 Архитектура

Libratus представлява конвейер от три модула, които функционират на три различни времеви мащаба — **извън линия** (преди мача), **онлайн** (при всяко вземане на решение) и **през нощта** (между отделните дни на игра) — и, за разлика от всички останали системи в тази глава, той не съдържа **никаква невронна мрежа** (Фигура 6.2).



Фигура 2: Libratus's three-module pipeline: an offline blueprint (top), the online nested safe-subgame solver that plays it (middle), and the overnight self-improver that grafts solved holes back into the blueprint (bottom, feedback arrow).

Модул 1 — планът (извън линия). Libratus първо компресираща приблизително 10^{161} решаващи точки на HUNL до около 10^{12} чрез два вида абстракция: *абстракция на действията*, която запазва само дискретно меню от размери на залозите (главно кръгли дробни и кратни на пота, взети от размерите, предпочитани от водещите конкурентни ботове, с няколко ранни размера, настроени чрез алгоритъм за оптимизация на параметри), и *абстракция на карти*, която групира стратегически сходни ръце. От решаващо значение е, че **не се използва абстракция на карти в първите два рунда на залагане** — достатъчно малки, за да се позволи пълна резолюция — а се групират само търнът и ривърът, и то дори там единствено в плана. След това тази абстрактна игра се решава чрез самообучение с усъвършенстван метод **Монте Карло за минимизиране на контрафактичното съжаление (MCCFR)**, който вероятно премахва клонове с много отрицателно съжаление, което представлява приблизително трикратно ускорение и също така тихо отстранява патология

на абстракциите с непълно извличане, при която няколко различни ситуации споделят стратегията на една кофа и „се борят“ за нея. Резултатът е **планът**: пълна, но неравномерна стратегия — детайлна в началото, груба в края — чиито стойности от късните рундове се използват не за игра, а единствено за *оценяване на стойността на достигане до под-игра*.

Модул 2 — вложено безопасно решаване на под-игри (онлайн). Libratus използва плана само в ранните кръгове. При достигане на третия рунд за залози — или всяка по-ранна точка, където останалата част от ръката е достатъчно малка — той изоставя грубия план за късната игра и вместо това **изгражда нова, по-финозърнеста под-игра без абстракция на карти и я решава в реално време** с високо оптимизиран CFR+ (бърз, детерминистичен вариант на CFR). Така всяка ръка се играе индивидуално в късната игра. Определящата характеристика е това, което се случва, когато противникът направи залог, който не фигурира в менюто на плана: вместо да го закръгли, Libratus решава нова под-игра, която *съдържа точно този залог*, и повтаря това за всяко последващо извънсписъчно действие — *вложено* решаване на под-игри. Този модул е обект на разглеждане в следващия подраздел.

Модул 3 — самоусъвършенстващият се (през нощта). Тъй като решаването в реално време се пропуска в първите два рунда (решаването толкова рано само по себе си би изисквало тежка абстракция), нестандартните залози на противника *там* все още се закръглят. Самоусъвършенстващият се стеснява този остатък между дните: той събира най-често използваните от противниците нестандартни размери на залозите, избира няколко от най-вредните пропуски, изчислява правилни игровитеоретични отговори на тях през нощта и присажда тези клонове в плана. Нарочно това **не е експлоатация на противника** — той никога не се опитва да моделира и наказва грешките на хората (което би изложило Libratus на контраексплоатация). Вместо това използва залозите на противниците единствено като насока *кои от собствените пропуски на Libratus да закръпи*, а закръпванията са универсални, подобрявайки играта срещу всеки бъдещ противник.

Класическите градивни елементи от по-ранните стъпки са на видно място: **абстракция на играта** и MCCFR изграждат **плана** извън линия, CFR+ извършва **решаването на под-игра** онлайн, а „търсене“ представлява **преизчисляване в реално време** за всяко решение. Забележителното отсъствие — липсата на **мрежа за стойности**, на **мрежа на стратегията** и на каквото и да е научено чрез **градиентен спуск** — е архитектурният подпис, който отличава Libratus от всяка друга система в тази глава.

6.3.3 Ключова иновация: вложено безопасно решаване на под-игри

Централният принос на Libratus е да направи **решаването на под-игра в реално време едновременно безопасно и вложено** — достатъчно силно, за да победи хората, и доказано неспособно да направи стратегията значително по-експлоатируема от плана, който усъвършенства. И двете прилагателни са важни и всяко от тях коригира конкретно **предварително убеждение** за неуспех.

Трудността е в това, че под-игра с непълна информация *не може да бъде решена изолирано*. В шаха, когато се достигне крайна игра, тя може да бъде решена самостоятелно, тъй като дъската е напълно известна. В покера правилната стратегия за една под-игра зависи от стратегиите в *други, недостижати* под-игри, защото именно те определят разпределението на вероятностите за скритите раздавания, които противникът може да държи тук. По-ранните решаващи системи в реално време заобикаляха това, като приемаха, че противникът

е следвал плана до този момент — **несигурно** решаване на под-игра — което противникът може лесно да накаже с отклонение, понякога произвеждайки усъвършенствана стратегия значително *по-лоша* от плана.

Libratus премахва това предположение с едно малко **приспособление**, *допълнената под-игра*. В корена на под-играта на противника се предоставя избор за всяко раздаване, което той може да държи: или да вземе фиксирана **алтернативна печалба** — оценката на плана за това колко струва това раздаване тук — или да *влезе* в детайлната под-игра и да я изиграе. Решаването на тази допълнена игра принуждава усъвършенстваната стратегия на Libratus да направи противника **не по-добре от тази оценка за всяко възможно раздаване**, което е точното значение на „безопасен“: локалната корекция не може да бъде експлоатирана спрямо плана, независимо какво прави противникът. Libratus изостря това с вариант, който авторите наричат **Estimated-Maxmargin** — той максимизира *най-малкия* резерв за безопасност във всички раздавания на противника — и с две усъвършенствания, които го правят силен, а не просто безопасен: той използва оценки на плана за стойностите на противника вместо консервативни **горна граница** (по-малко предпазлива игра) и *придава по-малка тежест на раздаванията, които противникът би могъл да държи само ако е допуснал грешка по-рано* (безсмислено е да се защитава срещу раздаване, което рационален противник никога не би довел до тази ситуация), освобождавайки капацитет за защита срещу раздаванията, които той реалистично може да има.

Комбинацията идва с гаранция, която е паралелна на тази на deepStack. Ако σ^* е най-малко експлоатируемата стратегия, която се различава от плана само в рамките на решените под-игри, а оценката на плана за стойностите на подиграта на противника има отклонение не повече от Δ , тогава експлоатируемостта на усъвършенстваната стратегия се подчинява

$$\text{exploitability}(\sigma_{\text{refined}}) \leq \text{exploitability}(\sigma^*) + 2\Delta.$$

Членът 2Δ представлява цената на несъвършените оценки на стойностите и е структурно аналогичен на члена $k_1\epsilon$ в deepStack — и двата определят колко една грешка в стойностите, подадени на решавача, може да повлияе върху експлоатируемостта в най-лошия случай. По този начин двете системи стигат до един и същи вид твърдение за коректност, но от противоположни посоки: ϵ при deepStack е грешката на приближението на невронна мрежа при границата на дълбочината; Δ при Libratus е грешката на оценката на таблична схема при границата на под-играта.

“Вложено” е втората половина. Вместо да разширява менюто за залози на плана (което би довело до експлозия на извънлинейното решаване), Libratus създава **отделен отговор на всеки нестандартен залог в реално време**: когато противникът направи залог с размер, който не е бил изброяван преди, той изгражда допълнена под-игра, чиято алтернативна печалба е най-доброто *в менюто* действие, което противникът би могъл да предприеме вместо това, решава я и го прави отново за всеки следващ нестандартен залог по време на ръката. В контролираните експерименти това превъзхожда предишния стандарт — закръгляне на залога до най-близкия размер в менюто — с повече от порядък по отношение на експлоатируемост в най-лошия случай (119 срещу 1465 mbb/g в докладвания тест с малки игри). И тъй като всяка под-игра се решава в движение, Libratus може също така да *променя собствените си размери на залозите* между ръцете — той

ги възмушава със случайна стойност от 0–8% при първото решаване — така че хората никога не се сблъскват с фиксирана цел за анализ. Това е точният механизъм, чрез който най-слабото място на парадигмата на абстракцията, преобразуването на действие, се отстранява от късната игра.

6.3.4 Ограничения, задънени улици и какво е подценено в статията

Както при deepStack, ясната тримодулна структура прикрива къде всъщност се крие инженерната работа — и откровеността относно ограниченията — с изключение на това, че тук голяма част от нея дори не е в статията в *Science*, а в придружителната статия в IJCAI и в лекционните бележки на авторите.

Най-разкривателното ограничение е, че **само планът не е свръхчовешки — той дори не побеждава предишния бот**. Срещу Baby Tartanian8, победителя от състезанието през 2016 г., суровият план на Libratus губи с 8 mbb/g; едва когато вложено решаване на под-игри беше включено, същата система спечели с 63 mbb/g. С други думи, предварителната стратегия е скеле и по същество цялото предимство на Libratus идва от търсене в реално време — резултат, който тихо преформулира цялата система и предвещава завъртането на областта към търсене при извод.

Няколко други важни детайла си струва да се отбележат. **Преобразуването на действие не е премахнато, а само ограничено**: в първите два рунда на залагане Libratus все още закръгля залозите на опонента извън менюто, а целият модул за самоусъвършенстване съществува, за да работи върху този остатъчен проблем, по три „дупки“ на нощ, без никога да го отстранява напълно. **Безопасното решаване на под-игри трябва-ше да бъде преработено, за да стане изобщо приложимо** — в продължение на три години се смяташе за непрактично, тъй като в директен сблъсък учебниково-безопасното решаване губеше от теоретично необоснования *небезопасен* вариант; използването от страна на Libratus на стойностни *оценки*, а не консервативни *горна граница*, заедно с намаляването на значението на ръцете, които опонентът би държал само по грешка, направи безопасността конкурентна и дори тогава Libratus умишлено използва *небезопасен* метод за решаване веднъж, при първото си влизане в третия рунд, защото там е по-евтин и емпирично достатъчно добър. Изборът на **Estimated-Maxmargin** сам по себе си жертва малко теоретична чистота за сметка на сила: използването на оценки вместо горна граница може по принцип да повиши експлоатируемостта *над* тази на плана, ограничена единствено от 2Δ на теоремата. А абстракцията с непълна памет в основата означава, че няколко различни ситуации споделят една стратегия и „се борят“ за нея — известно изкривяване, което подрязването, базирано на съжаление, само частично смекчава. И накрая, **изчислителната мощ почти отсъства от основната статия**: водещата публикация по същество не дава никакви данни, а истинската цена (следваща) излиза наяве едва във вторичните източници, както и фактът, че **кодът никога не е бил публикуван**, оставяйки независимата проверка да се основава единствено на публикувания псевдокод.

6.3.5 Изчислителна мощност и достъпност

Където сметката на deepStack се плаща почти изцяло извън линия, а играта му е евтина, Libratus е **скъп и в двата края**. Проектът е консумирал приблизително **25 милиона часове ядро на процесор** на свръхкомпютъра Bridges в Pittsburgh Supercomputing Center за една година — от които около 6 милиона са отишли за изграждане и решаване на плана, около 3 милиона за решаване на под-игри в реално време по време на мача,

около 3 милиона за самоусъвършенстващия се модул и останалите ~13 милиона за експериментални опити и оценка. Оперативно планът е работил като е заемал приблизително 195 възела за един до осем седмици наведнъж; всяко **решаване на под-игра в реално време е използвало около 50 възела и е отнемало порядък на десетки секунди**; самоусъвършенстващият се модул през нощта е работил върху до няколкостотин възела в продължение на часове; а стратегиите и моментните снимки са консумирали около 2,6 петабайта дисково пространство. **Не е имало gPUs и обучение на невронна мрежа никъде** — всеки час ядро е CFR или абстракция.

Формата на тази сметка носи поуката. Libratus не постига евтино внедряване чрез еднократно скъпо обучение, както прави deepStack; неговата *изчислителна мощност по време на игра* сама по себе си е в мащаба на свръхкомпютър, защото силата произтича от решаването на нова под-игра при почти всяко късно решение. От гледна точка на достъпността това направи Libratus, както беше внедрен, по същество невъзможно да бъде възпроизведен извън голям център за свръхкомпютри както по време на обучението, така и по време на играта — много по-висока летва от агента на deepStack с една GPU — а липсата на публикуван код подсили това. Основните техники са независими от приложението и работят значително по-евтино на съвременния хардуер и решаващите програми днес, но артефактът от 2017 г. беше демонстрация на това, което е възможно при наличието на изобилна изчислителна мощност, а не на това, което е широко достъпно.

6.3.6 Силни страни и ограничения

Сигналната сила на Libratus е **решителна, строго демонстрирана свръхчовешка игра**. Той е първият изкуствен интелект, който побеждава водещи специалисти по HUNL в дълъг мач — 147 mbb/g за 120 000 ръце при 99,98% значимост, като побеждава всеки от четиримата професионалисти индивидуално — и, за разлика от deepStack, той също така побеждава **предходния най-добър изкуствен интелект за покер в директен сблъсък** с 63 mbb/g, отделяйки своята сила от въпроса за човешкото умение. Основната му техника — **вложено безопасно решаване на под-игри** — е едновременно **доказуемо безопасна** (2Δ границата) и **емпирично с порядък по-малко експлоатируема** от преобразуването на действие, което заменя, като не използва **човешки данни и никакви специфични за домейна знания**. Модулният дизайн сам по себе си е принос: евтин план, решаващ в реално време, който го коригира там, където е необходимо, и нощен цикъл, който закръпва собствените му слабости — всеки модул компенсира недостатъка на друг — като цялата система е с **устойчивост на първо място**, отказвайки да експлоатира опонентите, за да не бъде контраексплоатиран.

Ограниченията са също толкова ясни и подготвят останалата част от главата. Libratus е **само за игра за двама с нулева сума**; неговите гаранции за безопасност се опират на тази структура. Той остава **абстрактно базиран** и все още **преобразува залози извън менюто в ранните рундове**, което представлява експлоатируема слабост, която самоусъвършенстващият се агент единствено стеснява. Той не използва **никакво невронно обобщаване**: нищо не се прехвърля между ситуациите, планът е огромен (петабайти от съхранена стратегия), а времето за игра е в **мащаба на свръхкомпютър**, а не едно GPU, както беше необходимо на deepStack. Компромисът на estimated-Maxmargin допуска малко, ограничено повишаване на експлоатируемостта в името на по-голяма сила; едно небезопасно решаване се промъква в конвейера и — тъй като както хората, така и изкуственият интелект се адаптираха по време на мача — дори значимостта в заглавието е,

стриктно казано, „сякаш-независима“ стойност, въпреки че предимство от 147 mbb/g за 120 000 ръце не оставя никакво реално съмнение.

6.3.7 Наследство и съвременна значимост

Извадете абстракционния механизъм и трайната идея на Libratus е **търсене в реално време, насложено върху груба предварително изчислена стратегия, направено безопасно при скрита информация**: решавайте локалната ситуация точно в момента на вземане на решение, в съответствие с евтин глобален план, и реагирайте на това, което опонентът действително прави, а не на закръглено приближение на него. Тази идея не само оцеля — тя се превърна, заедно с непрекъснатото пререшаване на deepStack, в половината от основата, върху която е изградена останалата част от главата. Вложеното безопасно решаване на под-игри на Libratus е пряк предшественик на търсенето в реално време в **Pluribus** (2019), то е обединено с пререшаването на deepStack чрез теорията за **ограничено по дълбочина решаване**, публикувана от авторите през следващата година, и същият модел „усъвършенствайте глобална оценка на стойността с локално решаване“ се появява отново, сега с *научени* стойности, заменящи табличната схема, в **ReBeL** (2020).

Libratus е също така, в точен смисъл, **високата граница на парадигма, която след това полето изостави**. Той постигна свръхчовешка игра **без никакво дълбоко обучение** — чиста абстракция и CFR — и въпреки това последващите системи са всички невронни, защото ръчно създадената абстракция и петабайтовите планове нито се обобщават между ситуациите, нито се мащабират отвъд двама играчи. Честната оценка е, че Libratus е *надминат като архитектура, но оправдан като теза*: неговият залог, че **търсенето в реално време има по-голямо значение от по-голяма предварително изчислена стратегия**, беше напълно точен, дори когато залогът му върху табличната абстракция беше надминат. Тази първа теза оттогава се превърна в централна тема на границата на изкуствения интелект — наблюдението, че добавянето на търсене във време на тестване струваше далеч повече от мащабирането на извънлинейното изчисление, е ранен, конкретен пример за аргумента „изчислителна мощност по време на тестване“, който сега се прави за модели с възможност за мислене, а разделянето план-тогава-търсене на Libratus предвещава съвременната рецепта предварително обучение и търсене.

Няколко подсистеми остават директно използвани: **безопасно решаване на подучастъци / ендшпил** като начин за локално усъвършенстване на глобална стратегия *без да се нарушават нейните гаранции*; **подпомагащият подигра** — чистата конструкция „вземете известна алтернативна стойност или влезте в подучастъка“ за закрепване на локално решение към глобална оценка на стойността; **подрязване, базирано на съжаление** за мащабиране на CFR; и моделът на **самоусъвършенстващ се**, който използва наблюдавана игра, за да открие и *поправи собствените си слабости*, вместо да експлоатира съперника. Това, което наистина е остаряло, е заобикалящата го структура — ръчно настроената **абстракция на действията** и картите, много-петабайтовата **таблична схема** и внедряването с клъстер по време на игра — всички от които се разтварят чрез научените представяния на по-късните системи. За тази теза конкретно Libratus допринася с две основи: неговата граница на експлоатируемостта при безопасно решаване на подучастъци е шаблон за **безопасна експлоатация при грешка в стойността (Принос 2)**, а умишленият му отказ да експлоатира — „поправете собствените си слабости, не моделирайте съперника“ — е точният контрапункт, спрямо който може да бъде дефинирана *ограничена, целенасочена адаптация към съперника* (Принос 1).

6.4 Pluribus (2019)

Libratus реши двуигровия No-Limit Hold'em, но неговите гаранции за безопасност — и всъщност самото значение на „решаването“ на играта — се основаваха на структурата на игра с нулева сума за двама играчи: там равновесието на Наш е непобедимо и решаването в реално време може да бъде доказано *сигурно* спрямо него. Холдемът, както хората всъщност го играят обаче, събира шестима. Pluribus (Brown & Sandholm, 2019; Carnegie Mellon и Facebook AI) се изправи директно пред многопотребителския въпрос: **какво се случва с рецептата план-плюс-търсене в реално време, когато премахнете двуигровия патерица и седнете на маса за шестима — обстановка, в която равновесието на Наш не е нито уникално, нито ефективно изчислимо, нито дори гаранция, че няма да загубите?** Отговорът му беше емпиричен и категоричен. В два формата — петима професионалисти, седнали с едно копие на Pluribus, и един професионалист срещу пет копия — той победи въртящ се състав от тринадесет елитни професионалисти, няколко от които шампиони от Световните серии или Световния покер тур, печелейки около **48 мили-големи блайнда на игра** срещу петима души наведнъж (mbb/g, мерната единица за процент победи от предишните раздели — приблизително пет големи блайнда на сто ръце, решаващ шестпотребителски марж) при 95% статистическа значимост. И го направи след обучение в продължение на **осем дни на един 64-ядрен сървър за около \$150 облачни изчисления** — с порядък хиляда по-малко от това, което свръхкомпютърът зад Libratus консумираше.

С един поглед	Pluribus (2019)
Играчи	6 (шестмакс) — първият свръхчовешки изкуствен интелект в която и да е референтна игра с повече от двама играчи/отбора
Тип игра	6-max NLHE — шестпотребителски No-Limit Texas Hold'em (непълна информация; многопотребителски, <i>не</i> двуигрови с нулева сума)
План (офлайн)?	Да — цялостен план за играта, решен офлайн чрез Linear MCCFR; използва се <i>директно</i> само в първия рунд за залози, след това служи като скеле
Невронна компонента	Няма — изцяло табличен CFR / абстракция (без невронни мрежи никъде, както при Libratus)
Механизъм за търсене	Търсене в реално време с <i>ограничена дълбочина</i> с $k=4$ стратегии за продължение в листата (вложени <i>несигурни</i> решавания, пререшаване от началото на всеки рунд за залози); Linear CFR вътре в под-игрите
Абстракция?	Да — действия (1–14 размера на залозите в плана; 1–6 при търсене) + информационна абстракция (без загуби в първия рунд; с загуби в по-късните, по-фини при търсенето, отколкото в плана)
Също и пълна информация?	Не (само непълна информация)

С един поглед	Pluribus (2019)
Изчислителна мощ	Прочуто евтин: план ~12 400 ядро-часа / 8 дни / < 512 GB на един 64-ядрен сървър (~\$144); игра на 2 процесора (28 ядра), < 128 GB, без GPU, 1–33 s/решение
Ключова иновация	Свръхчовешка шестпотребителска игра чрез план + търсене с ограничена дълбочина със стратегии за продължение — спечели <i>емпирично</i> , с минимален бюджет, без гаранция за безопасност при n играчи

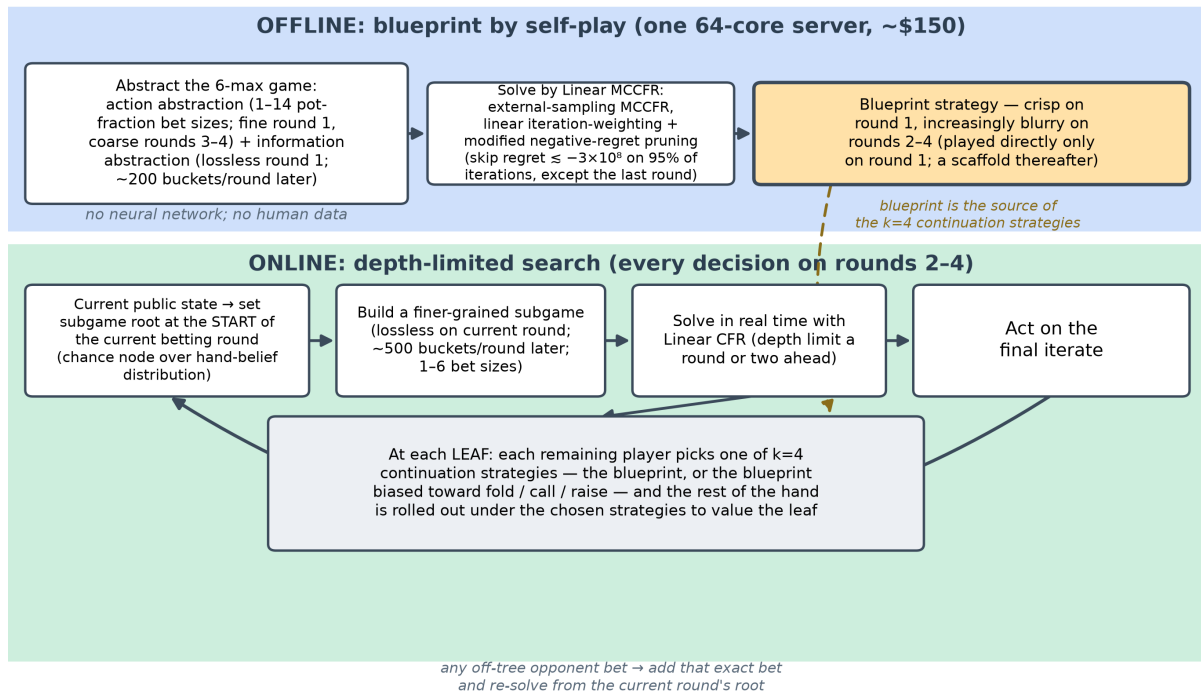
6.4.1 Пропуските, които запълни

Всеки свръхчовешки игрови изкуствен интелект преди Pluribus — дама, шах, Го и двете предходни покер програми — споделяше едно скрито предположение: двама играчи, нулева сума. В тази обстановка равновесието на Наш носи свойство, което го прави очевидната цел: всеки играч, който възприеме такова, е *гарантирано, че няма да загуби* в очакване, независимо какво прави противникът, и ако двама играчи независимо изчислят равновесия, техните стратегии все още се комбинират в равновесие. „Решаването“ на такава игра следователно *означава* приближаване на равновесие на Наш, а deepStack и Libratus бяха, по същество, два начина да направят точно това. Шестпотребителският покер разтваря всичко това. При повече от двама играчи намирането — дори приближаването — на равновесие на Наш е изчислително неразрешим проблем като цяло; обикновено има *много* равновесия; и, фатално, ако всеки играч независимо избере едно, получената съвместна стратегия може да не бъде равновесие изобщо, така че играчът може да *загуби*, докато играе безупречна равновесна стратегия. Brown и Sandholm илюстрират това с играта „Щанд за лимонада“, в която играчите се разполагат около пръстен: има безкрайно много равновесия и независимо избраните рядко се съчетават. Заключение е рязко — в многопотребителския покер равновесието на Наш *не е нито уникално, нито гаранция за безопасност*, а дефиницията на успеха в тази област от две десетилетия просто изчезва.

Pluribus затваря тази празнина, като променя целта и след това я постига. Вместо да преследва концепция на решение, на която не може да се довери, тя цели единствено *емпирично и последователно да побеждава елитни хора*, и предварително приема, че нейните алгоритми не носят гаранция за сближаване към равновесие извън играта за двама с нулева сума. Тази отстъпка е цялата същност — и това е точната празнина, която принос 2 на тази дисертация се стреми да запълни. Другата половина на празнината е механична. Libratus постигна свръхчовешка игра, като решаваше всяка под-игра в късната фаза *докрай* в реално време; с шестима играчи под-игрите експлодират експоненциално и решаването докрай става непрактично, докато алтернативата на deepStack — стойности на листата, обусловени от разпределения на убеждения — вече беше скъпа в двуигровия покер и се влошава с увеличаване на броя играчи. Следователно Pluribus се нуждаеше от търсене в реално време, което да гледа само малко напред и да спира — в игра, в която, както показват следващите раздели, ранното спиране е точно това, което простото търсене не може безопасно да направи.

6.4.2 Архитектура

Както и при Libratus, Pluribus се разделя на офлайн фаза, която изгражда план, и онлайн фаза, която извършва търсене — но балансът на силите между тях е обърнат и, отново както при Libratus, **няма невронна мрежа никъде** (Фигура 6.3).



Фигура 3: Pluribus’s two-phase architecture: an offline Linear-MCCFR blueprint (top) reused as the source of $k=4$ continuation strategies inside the online depth-limited search (bottom).

Извън линия Pluribus изчислява план за цялата игра чрез самообучение, започвайки от случайна игра и подобрявайки се чрез **метод Монте Карло CFR (MCCFR) с външна извадка**, който при всяка итерация избира един играч като „traverser“, симулира ръка и насочва този играч към действията, които са се представили по-добре. Той абстрахира играта по два начина — ограничен брой размери на залозите (*абстракция на действията*) и групи от стратегически сходни ситуации с карти (*информационна абстракция*) — но само грубо, и използва този план *директно* само в първия от четирите кръга на залагане, където абстракцията на действията се запазва по-фина и не се използва групиране на карти. Навсякъде другаде — флоп, търн, река — Pluribus извърля грубия план локално и **търси в реално време** по-фина стратегия, точно както направи Libratus, с една решаваща разлика: той не решава до края на играта. Той разглежда само един или два кръга напред до граница на дълбочината и спира, което прави шест играчи изчислително достъпни.

Запознатите градивни елементи са всички налице — абстракция и MCCFR изграждат плана, линейният CFR извършва онлайн решаването, а „търсенето“ е преизчисляване при всяко решение — и двата архитектурни отличителни белега са *отсъствия*. Няма **невронна мрежа** (системата е изцяло таблична, като Libratus) и, за разлика от Libratus, няма **самоусъвършенстващ се**: Pluribus никога не поправя своя план между сесиите, тъй като търсенето с ограничена дълбочина в три от четирите кръга вече извършва поправките. Две състав-

ки правят ранното спиране разумно и те са предмет на задълбочения анализ по-долу: внимателен отговор на въпроса какво изобщо може да означава „стойност на възел“ при скрита информация, и търсене, което започва не от текущото решение, а от *началото* на текущия кръг на залагане.

6.4.3 Ключова иновация: търсене с ограничена дълбочина с продължителни стратегии

Евристичното търсене в шах или Го работи, като гледа на фиксирано разстояние напред и отчита стойност от хоризонта при предположението, че и двете страни ще играят добре след това. Това предположение напълно се проваля при скрита информация, а Браун и Сандхолм илюстрират провала с еднократна последователна игра „камък–ножица–хартия“: ако търсещият приеме, че противникът ще играе равновесие (всеки ход една трета от времето) отвъд хоризонта, тогава всяко действие изглежда еднакво оценено на нула и търсещият може да се установи на „винаги играй камък“ — при което противникът преминава към „винаги хартия“ и истинската стойност рязко спада. Възел в игра с непълна информация следователно няма *една единствена стойност*: неговата стойност зависи от стратегията, която търсещият ще възприеме, което е именно това, което търсенето се опитва да определи.

Централният принос на Pluribus е начинът, по който все пак се определя стойност за възлите. Когато търсенето достигне границата на дълбочината, вместо да се фиксира една продължителна стратегия, **всеки играч, който все още е в ръката, избира измежду четири различни продължителни стратегии** за остатъка от играта и може да ги смесва. Четирите са умишлено опростени: самият план и три пристрастни копия, които умножават вероятността за *пас*, за *кол* и за *рейз* (всеки след това се ренормализира). Тъй като противникът винаги може да премине към продължителна стратегия, която най-много наказва търсещия, небалансирана стратегия — покер еквивалентът на винаги играене на камък — вече не се възнаграждава и търсещият се насочва към баланс. Тази идея първоначално беше доказана в двуигровия предшественик modicum (Brown, Sandholm & Amos, 2018), който победи двама бивши шампионски бота, докато работеше на 4-ядрен лаптоп с 16 GB памет — поразителен знак, че търсенето с ограничена дълбочина с продължителни стратегии може да замести свръхкомпютър.

Pluribus обобщава идеята от двама играчи на шестима и добавя фин, но важен обрат. В modicum само *противникът* избираше измежду продължителни стратегии, докато търсещият винаги следваше плана — подход, който е обоснован в **двуигрови игра с нулева сума**, но дава инициативата на противника и оставя търсещия колеблив и с ниска стойност. Pluribus позволява на **търсещия също да избира измежду продължителните стратегии**, което балансира играчите и, по думите на авторите, е „по-ефективно, по-лесно и по-елегантно“. Освен това той не пререшава от текущата точка за вземане на решение, а от *началото на текущия рунд за залози*, като фиксира единствено вече предприетите действия. Това „несигурно“ търсене — наречено така, защото, за разлика от Libratus, не носи гаранция за **експлоатируемост** — е по-евтино (повечето раздавания с шестима играчи се пасуват веднага, така че малко от тях изобщо се нуждаят от стратегия) и, тъй като започва непосредствено след случайно събитие с множество разклонения, на практика се оказва трудно за експлоатиране.

Това, което очевидно липсва от всичко това, е една *гаранция*. deepStack ограничи своята експлоатируемост до $k_1 \epsilon + k_2 / \sqrt{T}$, а Libratus — до 2Δ ; Pluribus не предлага такова ограничение и пропускането е принципно,

а не поради небрежност. Двигателят на CFR все още, във всяка крайна игра, свежда *средното съжаление* на всеки играч до нула,

$$\frac{R_i^T}{T} \rightarrow 0 \quad (\text{no-regret}),$$

но изводът, който доведе двамата предшественици от безсъжаление до безопасност — *безсъжаление играта се сближава до равновесие на Наш и едно равновесие на Наш не може да бъде победено* — важи само когато има двама играчи и играта е с нулева сума. При шестима играчи първото следствие отпада (самообучението не е задължително да се доближава до равновесие), а второто е безсмислено (едно равновесие не е непобедимо). Самите итерации, които *доказаха* deepStack и Libratus безопасни, следователно носят на Pluribus само емпирична сила: свръхчовешка на практика, без нищо сертифицирано. За да направи тези итерации достатъчно евтини за изпълнение с шестима играчи, Pluribus разчита на **линеен CFR** — претегляйки приноса на итерация t чрез t , така че лошите ранни итерации да се изчистват около три пъти по-бързо — и на **модифицирано отрицателно-съжаление подрязване**, което пропуска безнадеждни действия при повечето итерации (навсякъде с изключение на последния кръг); заедно с отложено разпределение на паметта, това са нещата, които намаляват сметката до един сървър.

6.4.4 Ограничения, неизследвани пътища и пропуснати детайли в статията

Както и при двамата предшественици, публикуваната статия е изчистеният разказ, а инженерната честност се намира в допълнителните материали — още повече тук, тъй като Pluribus е шестстранична статия в *Science*, чиято архитектура почти изцяло е прехвърлена към нейните допълнителни материали и към две придружаващи статии. Най-същественото признание е, че Pluribus използва **небезопасно** решаване на подигра — той предполага, че опонентите са играли стратегията, която той изчислява за тях — което, както авторите заявяват ясно, „няма теоретични гаранции за представянето дори в двуигрови игри с нулева сума и има случаи, в които води до силно експлоатируеми стратегии“. Съществуват безопасни алтернативи, но при директен сблъсък те се представиха по-зле, така че Pluribus взема емпиричната победа и смекчава риска само като винаги пререшава от началото на рунда за залози. Втора слабост е наследена и никога не е напълно затворена: в *първия* рунд за залози, твърде отдалечени от плана на опонента действия все още се **закръглят** чрез превод на действие — същата слабост, която самоусъвършенстващият се модул на Libratus съществуваше, за да преодолее — и Pluribus, тъй като няма такъв модул, просто живее с нея.

Няколко други детайла са от значение. Основните нововъведения на Pluribus **никога не са индивидуално премахвани**: авторите признават, че дисперсията на покера без лимит и цената на човешките изпитания правят „твърде скъпо“ измерването на приноса на всяко едно от тях, така че докладваните ускорения на компонентите (приблизително $3\times$ от линейния CFR, $2\times$ от модифицираното подрязване, повече от $2\times$ в паметта от отложеното разпределение) са оценки, а общото предимство не е разложено. Допълнението обаче разсейва едно изкушаващо погрешно схващане: приемането на *единствена* план-продължителна стратегия в листата — очевидното нещо за опитване и начинът, по който основните бележки понякога описват метода — е показано в двуигровия предшественик да *губи* от двамата шампионски бота (с 10 и 1 mbb/g), докато версията с четири продължителни стратегии *печели* (с 6 и 11); наборът от продължителни стратегии вър-

ши реална работа, а не е украшение. По-малки любопитни факти допълват картината: Pluribus играе своята **последна** търсене итерация, вместо обичайното усреднено време, за да избегне остатъчни лоши действия; той се научи да **изоставя „лимпинга“** по време на самообучението, но **„донкове“ много повече от хората;** и — както Libratus — неговият **код никога не е бил публикуван**, оставяйки само псевдокод за независима проверка, тъй като покерът се играе комерсиално.

6.4.5 Изчислителна мощност и достъпност

Историята за изчислителната мощност на Pluribus е тази, която повечето хора помнят, и тя наистина обръща с главата надолу своите предшественици. Планът беше обучен за **осем дни на един 64-ядрен сървър** за около **12 400 ядро-часа** и под 512 GB памет — приблизително **\$144** при спот цени в облака — а на масата Pluribus работи на **два CPU (28 ядра) и под 128 GB, без GPU на който и да е етап**, като взема от една до тридесет и три секунди за решение и играе около два пъти по-бързо от човек. В сравнение с останалите контрастът е почти комичен: AlphaGo използва 1920 CPU и 280 GPU, Deep Blue 480 персонализирани чипа, а Libratus около петнадесет милиона ядро-часа за изграждане на своя план и приблизително сто-CPU клъстер за игра. Pluribus достигна *по-труден* етап — повече играчи, по-голяма игра — за порядък от хилядна част от изчислителната мощност за обучение на Libratus. Докато DeepStack концентрира разходите си извън линия, а Libratus плащаше скъпо и в двата края, Pluribus е **евтин и в двата края**, и това, също толкова колкото резултатът с шестима играчи, е тезата на статията.

Това преформулира изцяло достъпността. За първи път една свръхчовешка покер система беше възпроизводима, по принцип, от един добре оборудван изследовател, а не от суперкомпютърен център — авторите я представят изрично като опровержение на опасението, че граничният изкуствен интелект в игрите ще принадлежи само на екипи с милиони долари хардуер. Свивът не е магия, а алгоритмичен: съчетаването на търсене с ограничена дълбочина (което авторите оценяват, че спестява поне пет порядъка спрямо решаване до края), линеен CFR и агресивно подрязване и пестене на памет. Единственото уточнение е, че самият артефакт остава затворен — без код — така че „достъпен“ описва *метода*, демонстриран в мащаб на лаптоп от Modicum, повече отколкото изтегляема програма.

6.4.6 Силни страни и ограничения

Силата на сигнала на Pluribus е просто в това, че е **първи**: първият изкуствен интелект, който постига свръхчовешко представяне във всяка широко призната референтна игра с повече от двама играчи или два отбора и в най-популярната форма на покера. Победата беше решителна и строга — **+48 mbb/g ($p = 0,028$)** срещу петима елитни професионалисти на масата и **+32 mbb/g ($p = 0,014$)** с пет копия срещу един-единствен професионалист — измерена с намалителя на дисперсията AIVAT (който намали дисперсията около девет пъти) в десетки хиляди ръце, срещу тринадесет професионалисти, всеки от които е спечелил над един милион долара и които имаха дни да търсят слабости; процентът победи едва се колебаеше. По-късен реванш дори победи **Linus Loeliger**, широко смятан за най-добрия Six-Max Cash Player в момента. И, подобно на Libratus, Pluribus е с **устойчивост на първо място**: той играе фиксирана стратегия, никога не моделира и не се адаптира към опонентите и дори не знае техните самоличности, така че не може да бъде примамен в контраексплоатируема корекция — дисциплина, която, с **без човешки данни и без експертни познания**,

поддържа резултата чист.

Ограниченията са именно това, от което се отказва, за да стигне дотук. Pluribus **няма абсолютно никаква гаранция за безопасност** в шестпотребителската обстановка — няма сходимост към равновесие на Наш, няма граница на експлоатируемостта — така че неговото свръхчовешко състояние е *емпиричен* факт за тринадесет силни човека в десетки хиляди ръце, а не теорема; достатъчно координирана маса или просто различна игра не носят никаква увереност. Той разчита на **несигурно** търсене, все още **закръгля залозите извън дървото в първия рунд**, и остава **изцяло табличен и абстрактно-базиран**, без нищо научено, което да обобщава в различни ситуации — планът е гигантска таблица за търсене, а не модел. Авторите добавят, че целият подход може да не оцелее там, където играчите могат да **комуникират и се стоварят**, което покерът до голяма степен забранява. И неговият процент победи от 48 mbb/g в шест ръце, макар и решителен, не е същата „валута“ като 147 на Libratus в един на един — различна игра, петима опоненти, по-висока дисперсия. Повечето от тези пропуски са поправени от по-късни системи; този, който тази дисертация откроява — **че многопотребителският успех дойде без никаква гаранция за безопасност** — е изричната цел на принос 2.

6.4.7 Наследство и съвременна значимост

Две идеи надживяват спецификите на покера. Първата е техническа: **търсене с ограничена дълбочина, направено коректно при скрита информация чрез даване на всеки възел малко меню от избираеми стратегии за продължаване вместо една-единствена стойност** — чист начин да се погледне само малко напред в игра, където наивно не можеш. Втората е икономическа и областта я възприе най-дълбоко: Pluribus е каноничната демонстрация, че **по-труден проблем може да бъде решен с хиляда пъти по-малко изчислителна мощност чрез по-добри алгоритми и търсене във време на вземане на решение**, вместо повече хардуер. Ноам Браун оттогава посочи точно това — че добавянето на търсене в реално време е струвало толкова, колкото огромното мащабиране на предварително изчислената стратегия — като ранен, конкретен пример за аргумента „**тест-тайм изчислителна мощност**“, който сега е централен за модели с възможност за мислене като o1. В архитектурната дъга на тази глава Pluribus е точката, в която *барьерата на броя играчи* пада; *барьерата на обобщаването*, която той оставя изправена — всичко все още е табличен и ръчно абстрахиран — е това, което **Recursive Belief-based Learning** и **Student of Games** разрушават след това, като заменят плана с научени стойности на състоянието на убеждението.

Няколко от частите на Pluribus остават директно използвани: конструкцията **продължителна стратегия (многозначна листовка)** за стабилно търсене с ограничена дълбочина във всяка среда с непълна информация или многоагентна среда; **Линеен (дисконтиран) CFR**, вече стандартен ускорител на сходимостта и отправна точка за по-късни дисконтирани варианти на CFR; **подрязване, базирано на съжаление** за мащабиране на CFR; **AIVAT** като нискодисперсностна оценка; и обработката *добавяне на действието и пререшаване* на залозите извън дървото, наследена от Libratus. Това, което е заменено, е заобикалящата таблична конструкция — ръчно настроената абстракция и планът под формата на таблица за търсене — които научените представяния на ReBeL и SoG разтварят, заедно с *несигурното* търсене, което тези методи със състояние на убеждението правят ненужно. Честната оценка отразява тази на Libratus: **заменена като внедрена архитектура, но оправдана и изострена като теза**. За тази дисертация по-специално Pluribus е ключовият камък на

принос 2 — емпиричното доказателство, че методите търсене по Наш и търсене *работят* в N-потребителски игри с непълна информация, като същевременно предлагат **абсолютно никаква гаранция за безопасност**, което е точно пропускът, който една теория на многоагентната безопасна експлоатация трябва да затвори.

6.5 ReBeL (2020)

Pluribus преодоля бариерата на броя играчи, но го направи, като остана точно това, което беше Libratus — гигантска, ръчно абстрахирана таблица за търсене без нищо научено, което да се прехвърля от една ситуация в друга — и постигна победата си с шестима играчи, като изцяло се отказа от безопасността, разчитайки на *небезопасно* търсене без никаква гаранция за експлоатируемост. ReBeL (Brown, Bakhtin, Lerer & Gong, 2020; Facebook AI Research) прави крачка назад от шестима към двама играчи и задава обратния въпрос: **какво ще стане, ако вместо ръчно да създаваме абстракция и предварително да изчисляваме план, пуснем AlphaZero — обучение чрез самоигра с подкрепление плюс търсене, както по време на обучението, така и при тестване — в игра със скрита информация, възстановявайки гаранциите, които Pluribus отхвърли, като същевременно изцяло премахнем абстракцията?** Неговият отговор, *рекурсивно обучение на основата на убеждения*, е първият алгоритъм, който прави обучението с подкрепление и търсенето доказуемо коректни в игри с непълна информация. Трикът е да се преформулира играта като „игра с пълна информация“ върху **публични състояния на убеждения** — вероятностни разпределения за това какво може да държи всеки играч, предвид общото знание — върху които функциите за стойност и политика са добре дефинирани; цикъл в стил AlphaZero след това обучава невронна мрежа за стойност (и политика) върху тези състояния, като минимизирането на контрафактичното съжаление (CFR) решава под-игри с ограничена дълбочина на листата. ReBeL доказуемо се сближава към равновесие на Наш във всяка игра за двама с нулева сума, побеждавайки топ човешкия специалист в хедс-ъп Донг Ким със 165 mbb/g (мили-големи блайнда на игра, мерната единица за резултат от предишните раздели) в рамките на 7 500 ръце, като използва *значително по-малко* експертни познания от всеки предишен покер изкуствен интелект — и, за разлика от затворените Libratus и Pluribus, неговата реализация (за Liar’s Dice) беше **отворена**.

С един поглед	ReBeL (2020)
Играчи	2 (хедс-ъп) — <i>гаранциите</i> са за игра за двама с нулева сума; алгоритъмът се обобщава за повече играчи, но без гаранциите
Тип на играта	Общи двуигрови игри с непълна информация; оценяван върху hUNL poker + лъжливи зарове (и ендгейм холд’ем на ход). Редуцира се до алгоритъм в стил AlphaZero при игри с пълна информация
План (офлайн)?	Няма съхранен план — офлайн продуктът е <i>научена мрежа за PBS стойности (+ политика)</i> от самообучение, а не таблица на стратегиите

С един поглед	ReBeL (2020)
Невронна компонента	Мрежа за PBS стойности + (по избор) мрежа на PBS стратегията; MLP (GeLU/LayerNorm), 6×1536 скрити слоя за покер, вход = убеждение върху всеки от 1 326 ръце на играча + борда + пота + флаг за залог
Механизъм за търсене	CFR (CFR-D / CFR-AVG; също FP/FLOP), решаващ ограничена по дълбочина под-игра, кореняща се в PBS, като научената мрежа за стойности доставя (зависими от итерацията) стойности на листата — както по време на обучението , така и при тестване
Абстракция?	Няма — няма абстракция на карти/информация (със загуби или без); мрежата за стойности я замества. Запазва се само малко (≤ 9) ръчно избрано меню от размери на залозите, като залозите извън дървото се добавят в реално време
Също и пълна информация?	Не (представен/оценяван като игра с непълна информация), с уточнението, че <i>дегенерира</i> до търсене в стил AlphaZero, ако частната информация бъде премахната — пълното обединяване е претенция на Student of Games
Изчислителна мощ	Обучение на GPU: пълният hUNL използва ~ 90 DGX-1 възела $\times 8$ V100 GPU за генериране на данни от самообучение (в контраст с CPU-only $\sim \$150$ на Pluribus); CFR на един CPU поток; игра < 2 s/ръка, ≤ 5 s/решение
Ключова иновация	Публични състояния на убежденията + цикъл на самообучение в стил AlphaZero, обучаващ PBS мрежа за стойност/политика с CFR, работещ в пространството на убежденията: първото <i>коректно</i> RL+търсене за игри с непълна информация, възстановяващо доказуема 2p0s гаранция на Наш без абстракция или план

6.5.1 Затворената празнина

Предишните три системи се сходилих към една и съща рецепта — предварително изчислен план плюс търсене в реално време — а Pluribus я разшири до шестима играчи, но всяка от тях, в основата си, беше *табличен* обект: стратегия или план, съхранени върху ръчно проектирана абстракция, без нищо научено, което да се обобщава към ситуация, която никога не е била изрично решена. Само deepStack имаше невронна компонен-

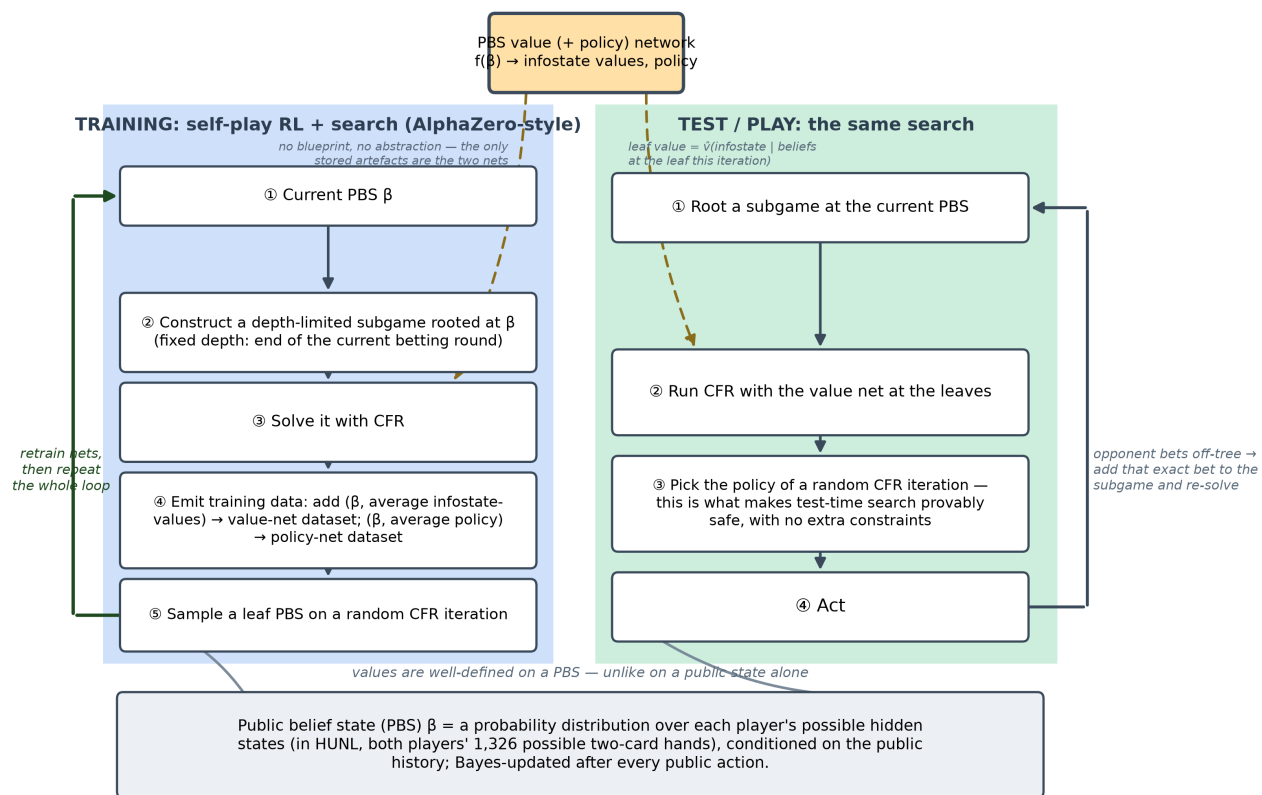
та, но дори и той разчиташе на ръчно извлечени признаци, клъстеризация в 1000 кофи на входа на мрежата и абстракция на ривъра. Междувременно най-успешният парадигма във всички игрови изкуствени интелекти — съюзът на AlphaZero между обучение чрез самоигра с подкрепление и търсене, който научава собствената си оценка от нулата и я използва както за обучение, така и за игра — беше *недостъпен* за игри с непълна информация. Отвореният въпрос, на който ReBeL отговаря, е този, който Ноам Браун нарича „свещения граал“ на областта: **може ли рецептата на AlphaZero да бъде накарана да работи коректно в игри със скрита информация?**

Причината, поради която това не беше възможно преди, е фина и представлява концептуалната същност на цялата глава. AlphaZero предполага, че всяко състояние има една единствена добре дефинирана стойност: шахматна позиция струва толкова, колкото струва, независимо колко често достигате до нея. Скрытата информация унищожава това предположение. Статията го прави нагледно с модифициран камък-ножицартия, в който ножицата печели (или губи) двойно: равновесието е да се хвърля камък и хартия 40% от времето и ножица 20%, и при това равновесие всяко действие има очаквана стойност нула. Ако Играч 1 извърши еднопластово търсене с придвиждане, което работи в шаха — замествайки равновесната стойност на листата — всяко движение изглежда еднакво добро, така че търсещият може да се спре на „винаги камък“, след което противникът преминава към „винаги хартия“ и истинската стойност на камъка се срива от 0 до -1 . **В игра с непълна информация стойността на едно действие зависи от вероятността, с която то се изиграва**, така че състояние, дефинирано само от последователността на действията, няма уникална стойност — и търсенето в стил AlphaZero е просто некоректно. Pluribus беше закрил симптома със стратегии за продължение по избор на листата; ReBeL лекува болестта, като променя това какво е едно „състояние“. Той предефинира състоянието така, че да включва разпределението на вероятностите върху скритата информация — *общодостъпно състояние на убеждението* — върху което, както показват следващите раздели, стойностите отново стават добре дефинирани и цялата машина на AlphaZero може да бъде прехвърлена. При това той също така премахва двете патерици, на които все още се опираха deepStack и системите за абстракция: ръчно създадената абстракция и предварително изчисления план изчезват, заменени от функция на стойността, научена единствено от самообучение.

6.5.2 Архитектура

ReBeL е най-добре да се възприема като **AlphaZero за непълна информация**: цикъл на самообучение, който обучава невронни мрежи за стойности и стратегии, при който „търсенето“, използвано както по време на обучение, така и при игра, представлява CFR solving на ограничена по дълбочина под-игра — но всичко се извършва върху *общодостъпни състояния на убеждението*, а не върху сурови игрови състояния, като оценителят на възел е научена мрежа за стойности, вместо разгръщане (Фигура 6.4).

Цикълът има три основни компонента. **Обучението чрез самоигра** е движещата сила: започвайки от кореново PBS, ReBeL конструира ограничена по дълбочина под-игра, решава я, записва решените стойности и стратегия като цели на обучението, избира възел, който да стане следващото кореново PBS, и продължава до края на играта — след това преобучава мрежите и повтаря процеса, по същия начин както AlphaZero редува самообучение с обучение на мрежата. **Търсенето** представлява вътрешното решаване: ReBeL изпълнява CFR (конкретно CFR-D, „CFR с декомпозиция“, или неговия вариант CFR-AVG) върху ограничената



Фигура 4: ReBeL's AlphaZero-style loop: self-play training (left) and test-time play (right) share one PBS value/policy network, both solving depth-limited subgames rooted at the public belief state (shared definition box, bottom).

по дълбочина под-игра и на всяка итерация на CFR задава стойността на всеки възел чрез заявка към **научената pBS value мрежа** — така че стойностите на възлите се променят от итерация до итерация, което е именно това, което гарантира коректността на търсенето при наличие на скрита информация. **Невронните мрежи** възплъщават научената интуиция: мрежа за стойности, която съпоставя PBS със стойностите на възможните ръце на играча, и незадължителна мрежа на стратегията, която осигурява топли стартове на CFR, намалявайки броя необходими итерации.

Архитектурните особености се изразяват в *липсата* на елементи, характерни за по-ранните системи. Няма **предварително изготвен план** — единствените резултати от обучението са двете мрежи. Няма **абстракция** от какъвто и да е вид: докато DeepStack, Libratus и Pluribus групираха стратегически сходни ръце, ReBeL изчислява отделна стратегия за всяко информационно състояние, като подава на мрежата единствено суровото разпределение на убежденията върху възможните 1326 ръце и на двамата играчи, плюс дъската, пота и един флаг „дали някой е заложил този рунд“. А **сFR търсене** работи на **един CPU поток** без никаква абстракция; основната изчислителна мощност се използва за **gPU самообучение**, което обучава мрежата за стойности (балансът е разгледан под *Изчислителна мощност*). Класическите градивни елементи от по-ранните стъпки все още са видими — **сFR търсене** (Стъпка 3) изпълнява търсенето, а **невронното приближение на стойността** (Стъпка 5) е оценителят на възел — но сега те са обединени в един цикъл самообучение плюс търсене, като **общодостъпното състояние на убеждението** (разделът по-долу) служи за представянето, което прави това обединение възможно.

6.5.3 Ключова иновация: общодостъпни състояния на убеждението и научени стойности в пространството на убежденията

Приносът на Recursive Belief-based Learning е един концептуален ход с три технически последствия. Ходът е да се спре търсенето върху **състояния** и да се започне търсене върху **убеждения за състоянията**.

Общодостъпното състояние на убеждението (PBS) е сърцевината на всичко това. Блогът на Meta AI дава най-ясната интуиция. Вземете игра с карти и я променете така, че играчите да не могат да виждат собствените си карти — само безпристрастен съдия може; на всеки ход играч обявява, за всяка карта, която *може* да държи, вероятността, с която би предприел всяко действие, и съдията избира хода за истинската карта на играча. Тъй като стратегиите на всички играчи се приемат за общоизвестни, всеки може да проследява, чрез правилото на Бейс, вероятността всеки играч да държи всяка възможна ръка. Тази модифицирана игра е **стратегически еквивалентна** на оригиналната, но тя не съдържа *частна информация*: нейното състояние — векторът от тези вероятности — е напълно наблюдаемо за всички. Recursive Belief-based Learning нарича това състояние общодостъпно състояние на убеждението: формално, **съвместно разпределение на вероятностите** върху възможните информационни състояния на играчите, дадено общите публични наблюдения. От решаващо значение е, че разглеждането на игрите с непълна информация като игри с непрекъснато състояние и пълна информация по този начин е стара идея (тя датира от работата върху кооперативни POMDPs); постижението на Recursive Belief-based Learning е, че за първи път я комбинира с обучение чрез самообучение в *състезателна* среда.

Първото следствие е, че **стойностите отново стават добре дефинирани**. Под-игра с непълна информация,

вкоренена в обикновено *публично състояние*, няма уникална стойност, защото нейната стойност зависи от скритите раздавания, които я достигат — което е точно причината евристичното търсене да се провали. Но под-игра, вкоренена в *общодостъпно състояние на убеждението*, има такава: в двуйгрови игри с нулева сума всяко общодостъпно състояние на убеждението β носи уникална стойност $V_i(\beta)$ с $V_1(\beta) = -V_2(\beta)$, дефинирана от това и двамата играчи да играят равновесие на Наш в под-играта от β . Това е именно свойството, на което AlphaZero разчита в шаха, сега възстановено за покера — и то *формализира и обобщава контрафактичните стойности при дадени убеждения на deepStack*, които бяха първият случай на функция на PBS стойност, но бяха обучени по много по-ад хок начин.

Второто следствие е, че **alphaZero loop** може да работи с **CFR като алгоритъм за търсене**. По принцип би могло да се стартира търсене в дърво на Монте Карло на AlphaZero директно върху играта на убежденията, но представянето на убеждението е много високодимензионно непрекъснато пространство (в малкия пример с карти от блога само пространството на действията има 156 измерения) и там търсенето с дърво е безполезно. Спасителният фактор е, че в двуйгрови игри с нулева сума тези представяния на убежденията са *вдълбнати* задачи за оптимизация, а CFR е по същество решаващ, базиран на градиент, за тях. Така ReBeL търси с CFR вместо с търсене в дърво на Монте Карло — единствената съществена разлика от AlphaZero. Техническа тънкость, която статията доказва (Теорема 1), е, че количествата, които CFR действително изисква — стойностите за всяко информационно състояние — са *сверхградиент* на вдълбнатата функция на PBS стойност, така че ReBeL обучава мрежа за стойности на информационни състояния, вместо един скалар за всеки PBS.

Третото следствие — и водещото заглавие — е, че цялото нещо е **доказуемо коректно**, възстановявайки гаранцията, която Pluribus се отказа да даде. Най-дълбокият резултат засяга *времето на изпитване*. По време на самообучение Recursive Belief-based Learning може да приеме, че стратегиите и на двамата играчи са общо знание, така че винаги знае истинското PBS; но срещу реален противник то не знае неговата стратегия и следователно не знае в кое PBS се намира, което наивно прекъсва търсенето. Елегантното решение на Recursive Belief-based Learning е да стартира *същия алгоритъм* по време на изпитване и **да действа според стратегията на произволно избрана итерация на CFR**. Статията доказва (Теорема 3), че това води до безопасно търсене — равновесие на Наш *в очакване* — без допълнителни ограничения, където всеки предишен метод за „безопасно търсене“ добавяше ограничения толкова скъпи, че никога не бяха използвани изцяло в състезателен бот. Конкретно, с мрежа за PBS стойности, чиято грешка е най-много δ и T итерации на CFR на под-игра, Recursive Belief-based Learning играе ε -равновесие на Наш — такова, което никой противник не може да експлоатира за повече от ε — с

$$\varepsilon = \delta C_1 + \frac{\delta C_2}{\sqrt{T}},$$

за специфични за играта константи C_1, C_2 . Структурата умишлено отразява тази на $k_1\varepsilon + k_2/\sqrt{T}$ от DeepStack — термин за грешка в стойността плюс обикновен термин за сходимост на CFR — но докато границата на DeepStack управлява експлоатируемостта на едно преизчисляване, тази на ReBeL управлява *сходимостта към равновесие на Наш на цялата игра*, и двата термина вече се мащабират с грешката на научената стойност δ : с подобряването на мрежата за стойности, $\delta \rightarrow 0$ и играта на ReBeL се доближават

до точно равновесие на Наш. Това е точният смисъл, в който ReBeL *се връща към двуигрови игри с нулева сума и възстановява теоретичните гаранции, които Pluribus се отказа да даде* — същият $1/\sqrt{T}$ гръбнак на CFR, който доказва безопасността на DeepStack и Libratus, сега обвит около стойност на състоянието на убежденията, научена чрез самообучение, вместо ръчно изградена абстракция.

6.5.4 Ограничения, задънени улици и какво статията описва недостатъчно

Recursive Belief-based Learning обръща обичайния модел на тази глава: *теорията* е в основния текст, а *инженерната честност* — по-специално точният смисъл, в който се използва „значително по-малко експертни познания“ — е преместена в приложенията.

Най-важният детайл е това, което ReBeL умишлено **отхвърля**, каталогизирано в Приложение D. Всеки предишен водещ изкуствен интелект за покер, включително deepStack, използваше **информационна абстракция**, за да групира стратегически подобни ръце; ReBeL не използва *никаква*, нито загубна, нито беззагубна, като изчислява уникална стратегия за всяко информационно състояние от суровото разпределение на убежденията. deepStack обучаваше своята мрежа за стойности върху *случайно генерирани* PBS, извлечени от ръчно настроен генератор; ReBeL генерира своите тренировъчни PBS изцяло от самообучение, твърдейки, че случайната извадка „би била като да се научи функция на стойността за Го чрез случайно поставяне на камъни върху дъската“ — и наистина показва (Фигура 2), че мрежа за стойности, обучена върху случайни убеждения, „се проваля в научаването на нещо ценно“. Предишните агенти предварително изчисляваха точни **таблици със стойността при отиграване докрай** и решаваха *до края на играта* на третия рунд за залози (използвайки **речна абстракция**, за да направят това постижимо); ReBeL не прави нито едно от двете, като винаги решава само до края на *настоящия* рунд и сам научава дори стойностите при отиграване докрай. Резултатът е честен, но с двойно острие: ReBeL трябва да научи *шест* „слоя“ от стойности, където deepStack се нуждаеше от три, което *увеличава* повърхността за **разпространение на грешка** — реална цена, приета в замяна на премахването на абстракцията. И не е без експертни познания: запазва ръчно избран списък с най-много осем или девет размера на залози (възпрепятствани по време на обучението), единствената отстъпка пред постижимостта, като залозите извън дървото се обработват чрез добавянето им към под-играта и **пререшаването ѝ**, à la Libratus и Pluribus.

Няколко други важни момента трябва да се отбележат. Чистите теореми стъпват върху **идеализации**: сходимостта в Теорема 2 предполага съвършен апроксиматор на функции, а гаранциите изискват стратегиите на играчите да са общо знание (резултатът от §6 премахва *време-на-тестване* версията на това предположение, но анализът на обучението все още идеализира мрежата). Вариантът, който агентът реално използва за ефективност — модифициран **CFR-AVG** — според самите автори *няма доказана теоретична обоснованост* („дали тази модифицирана форма на CFR-AVG е теоретично обоснована остава отворен въпрос“), въпреки че се представя добре в покера — позната празнина между доказания алгоритъм и внедрения. Гаранцията за безопасно търсене също зависи от избора на *случайна* итерация на CFR, която може да се окаже много лоша в ранните етапи; това се смекчава единствено чрез използването на Линеен CFR, който намалява тежестта на ранните итерации. И както при предшествениците си, по същество цялата архитектура, хиперпараметрите и най-важните данни за изчислителната мощност са представени в приложенията, а не в шестте страници основен текст — статията е, по дизайн, теоретична.

6.5.5 Изчислителна мощност и достъпност

Профилът на разходите на Recursive Belief-based Learning е огледален образ на този на Pluribus и контрастът е най-рязък в тази глава. Там, където интуицията на Pluribus беше план, решен чрез **CFR само с процесори на един 64-ядрен сървър за около \$150**, интуицията на Recursive Belief-based Learning е невронна мрежа и нейното обучение е в **кълъстерен мащаб на графични процесори**. Тясното място е генерирането на данни от самообучение — секвенциален CFR решава, чиято всяка итерация оценява всички възли листа чрез мрежата — така че статията паралелизира генерирането на данни върху „до 128 машини с по 8 gPUs всяка“, а пълният агент за HUNL в частност използва **около 90 възела DGX-1, всеки с осем 32 GB Nvidia V100 gPUs** (от порядъка на 700 gPUs), захранвайки една единствена тренировъчна машина с над 1750 епохи от по милиони примери всяка. Това е категорично сметка за обучение чрез дълбоко обучение, а не CFR сметка: самото CFR търсене се изпълнява на едно процесорно ядро. На масата обаче Recursive Belief-based Learning е евтин и подобен на deepStack — под две секунди на ръка средно и никога повече от пет секунди за решение, като предварителните под-игри са кеширани, за да бъде още по-бърз.

Историята с достъпността се разделя на две и това е най-ясният контраст със системите за покер от по-ранен период. От една страна *методът* на Recursive Belief-based Learning е най-общият и най-малко ръчно настроеният от четирите, и — решаващо — неговата **реализация беше с отворен код** (за „лъжливи зарове“), докато кодът както за Libratus, така и за Pluribus остана затворен. Това отворено издание е истинска стъпка към възпроизводимост, която системите на CMU никога не предлагаха. От друга страна, основният *покер* резултат *не* беше публикуван — авторите го задържаха с изричното основание, че Recursive Belief-based Learning „може да изчисли стратегия за произволни размери на стакове и произволни размери на залози за секунди“, което го прави готов инструмент за измама — а възпроизвеждането на този резултат в пълния му мащаб изискваше голям флот от V100, какъвто повечето групи не притежаваха. Така че Recursive Belief-based Learning е „отворен“ по начин, по който неговите предшественици не бяха, но отвореният артефакт е изследователската игра, а не свръхчовешкият покер бот.

6.5.6 Силни страни и ограничения

Силата на сигнала на Recursive Belief-based Learning е **общност с гаранция**. Това е първият алгоритъм, който прави парадигмата RL+Search на AlphaZero *доказуемо коректна* в игри с непълна информация, той доказуемо се сближава към равновесие на Наш във всяка **двуйгрови игра с нулева сума** и подкрепя теорията с резултати: побеждава предишните шампиони Slumbot (+45 mbb/g) и babyTartanian8 (+9 mbb/g), довежда **сондажа за най-добър локален отговор** до голяма загуба и побеждава топ човешкия специалист Донг Ким с 165 mbb/g в 7500 ръце — всичко това, използвайки *значително по-малко експертни познания от всеки предишен покер изкуствен интелект*, без **абстракция на карти** и най-общото представяне от четирите. Същият алгоритъм, непроменен, се сближава към **приблизително равновесие на Наш** в „лъжливи зарове“, демонстрирайки, че това е *рамка*, а не покер програма; в игри с пълна информация той елегантно се свежда до метод, подобен на AlphaZero. И той възстановява, за невронната ера, безопасността, която Pluribus трябваше да изостави — търсене, което е **доказуемо безопасен** по време на изпитване без допълнителни ограничения.

Ограниченията са точни и подготвят финалната система. Гаранциите на Recursive Belief-based Learning важат **само за двуигрови игри с нулева сума**: уникалната pBS стойност, изпъкналостта, която позволява cFR-as-Search, и доказателствата за коректност се основават на тази структура. Макар алгоритмът да е изпълняван и с повече играчи, теорията не го обхваща в тези случаи — „2 играча“ в оценката се отнася до *гаранцията*, а не до кода. Най-конкретната му стена на мащабиране е, че **пространството на убежденията нараства с количеството скрита информация**: входът на мрежата се мащабира с броя на инфосъстоянията в публично състояние, така че игри с голяма стратегическа дълбочина, но малко общо знание — статията посочва „слепец шах с разужнаване“ — раздуват представянето, а добавянето на играчи само увеличава пространството на убежденията. Алгоритмът предполага, че **точните правила на играта са известни** (разширение в стил MuZero към неизвестна динамика е посочено като бъдеща работа). Той наследява остатъчна **грешка на приближението на стойността δ** , която ограничава колко близо до равновесие на Наш всъщност играе, а най-силният му емпиричен резултат се основава на **един човешки противник** в 7500 ръце, вместо на многопроцесорно поле.

6.5.7 Наследство и съвременна значимост

Отстранете покера и трайната идея на ReBeL е представителна: **преобразувайте скритата информация в състояние на убеждението и пълната сила на обучение чрез самоигра с подкрепление плюс търсене се прехвърля**. Публичните състояния на убеждението вече са стандартният субстрат за коректно търсене в игри с непълна информация, а ReBeL е най-чистото изявление на „AlphaZero за непълна информация“ — търсене, вплетено както в *обучението*, така и в *играта*. Неговият пряк наследник е **Student of Games (2023)**, който пренася идеята за състояние на убеждението плюс коректно търсене по-нататък в единен алгоритъм, обхващащ игри с пълна и непълна информация; същата линия обучение чрез самоигра плюс търсене с научени модели също води до **CICERO (2022)**, агента на Meta за игра на Diplomacy на човешко ниво, който планира с научени модели в седем-игрови, със смесени мотиви, игра на естествен език далеч извън двуигровите гаранции на ReBeL. По-широко погледнато, ReBeL принадлежи към линията работа — deepStack, Libratus, Pluribus и сега ReBeL — която Ноам Браун посочва като ранен, конкретен пример за **изчислителна мощност във време на тестване**: изразходване на изчисления за търсене в момента на вземане на решение, насочвано от научена интуиция, вместо всичко да се пече в една предварително изчислена стратегия, същата теза, която сега е централна за моделите с възможност за мислене.

Няколко подсистеми остават директно използваеми. Самата **pBS representation** е най-важната — разпределение на убежденията върху скритите състояния на противниците при наличие на общо знание — и тя представлява естествената основа за базирано на убеждение моделиране на противника: Принос 1 на тази дисертация разширява pBS, така че да включва убеждения относно *типа стратегия* на противника, а не само неговите карти. **Обучената чрез самоигра функция на стойност/политика върху състояния на убеждението**, моделът на cFR-as-Search с научен оценител на възел, трикът за „безопасно търсене безплатно“ чрез произволни итерации и бързият търсач на равновесие **FLOP**, представен в статията, са всички преносими. Това, което е преодоляно, е основно ръчно изграденото скеле на DeepStack, разтворено от самообучението без абстракция на ReBeL, а след това и самият ReBeL като *обща* рамка, която Student of Games обхваща. Честната оценка е, че **ReBeL е жив стъпаловиден камък, а не крайна точка**: като внедрен покер

артефакт той никога не беше пуснат и сега е засенчен, но като идея — публични състояния на убеждението плюс AlphaZero-style RL и търсене — той е основополагащ и много жив. За тази дисертация конкретно той действа в две посоки: от една страна е отправната точка за по-богатото състояние на убеждението в Принос 1, а от друга — доказателството, че коректността, изоставена от Pluribus, *може* да бъде възстановена — макар все още не и при повече от двама играчи, което е точно границата, която Принос 2 се стреми да премине.

6.6 Student of Games / SoG (2023)

ReBeL възстанови коректността за игра за двама с нулева сума покер и изостави както абстракцията, така и плана — но той остана метод с непълна информация, който просто *се свеждаше* до AlphaZero в игри с пълна информация, вместо да ги обедини, решаваше *фиксирана* ограничена по дълбочина под-игра и обвързваше своето търсене по време на тестване със своята процедура за обучение. Student of Games (Schmid et al., 2023; Google DeepMind, с партньори от Университета на Алберта и EquiLibre Technologies в Прага) затваря последната празнина в главата, като задава най-амбициозния въпрос от всички: **какво ще стане, ако един единствен алгоритъм, който се учи чрез самообучение без човешки данни, може да играе шах, Го, покер един на един и Scotland Yard — разширявайки собственото си дърво за търсене според нуждите и оставайки доказуемо коректен както за игри с пълна, така и с непълна информация?** Неговият отговор обединява двете големи линии на главата — по думите на първия автор Мартин Шмид, това е „AlphaZero и DeepStack в един голям унифициран алгоритъм“. Двигателят е **Growing-Tree контрафактично минимизиране на съжалението (GT-CFR)**: търсене по всяко време, което изгражда дървото на играта постепенно, водено от мрежа за политики, с научена мрежа за стойности, оценяваща листата, които все още не са разширени. Върху поддървета с пълна информация GT-CFR се държи като MCTS на AlphaZero; върху тези с непълна информация — като CFR; и една единствена теорема за коректност покрива и двете. Системата достига **силно аматьорско до професионално ниво в шах и Го, побеждава Slumbot — най-силният публично достъпен бот за хедс-ъп безлимит покер — и надвишава най-съвременното ниво на агент за Scotland Yard**, като същевременно е доказано, че се сближава към минимакс-оптимална игра с нарастването на изчислителната мощ и точността на мрежата. Той споделя няколко автори от DeepStack (Bowling, Moravčík, Burch, Schmid) — линията от Алберта, която започна тази глава, тук в контраст с произхода на ReBeL от Facebook/FAIR — и първоначално се появи през 2021 г. под името *игращ на игри*, преди да бъде преименуван за своята публикация в *Science Advances* през 2023 г.

С един поглед	Student of Games (2023)
Играчи	2 (игра за двама с нулева сума) — гаранциите <i>и</i> оценката са за 2p0s (детективът в Scotland Yard се брои като един отбор); основната формализация е по-обща, но равновесието на Наш е „по-малко смислено“ извън 2p0s

С един поглед	Student of Games (2023)
Тип игра	Както с пълна, така и с непълна информация — единствената унифицирана система: шах + Го (пълна) и HUNL покер + Scotland Yard (непълна)
План (офлайн)?	Не — офлайн продуктът е научена мрежа за стойности и политики, а не съхранена таблица на стратегиите (както в ReBeL)
Невронна компонента	Една единствена контрафактична мрежа за стойност и политика (CVPN) : една мрежа извежда контрафактични <i>стойности</i> по инфо-състояния и предварителна <i>стратегия</i> , за всички етапи на играта
Механизъм за търсене	GT-CFR — редува фаза на актуализация на съжалението в CFR (стойности на CVPN в листата) с pUCT expansion phase в стил AlphaZero, който <i>разширява</i> публичното дърво; работи вътре в продължително пре-решаване, както по време на обучение , така и по време на тестване
Абстракция?	Няма от вида на картите/информацията (CVPN я заменя); запазва само малко <i>рандомизирано залагане</i> (действие) меню в покера ($\approx 20\,000 \rightarrow 4-5$ действия), както ReBeL
Също и за игри с пълна информация?	Да — отличителният ред; единствената система в главата, демонстрирана върху игри с пълна информация, със <i>същия</i> алгоритъм и коректност, покриваща и двата класа
Изчислителна мощ	Обучена на TPU, умишлено съпоставена с бюджета на AlphaZero (Google TPUv4; Го е най-скъпата); докладвано <i>относително</i> спрямо AlphaZero, без една единствена сума в долари; търсенето е $O(kT^2)$ ($O(T)$ за игри с пълна информация); пълният агент/код не беше публикуван
Ключова иновация	Growing-Tree CFR + правилно самообучение: един алгоритъм за самообучение с търсене, коректен и за двата класа игри , който разширява дървото си постепенно с CVPN в листата и доказуемо се сближава към равновесието на Наш

6.6.1 The gap it closed

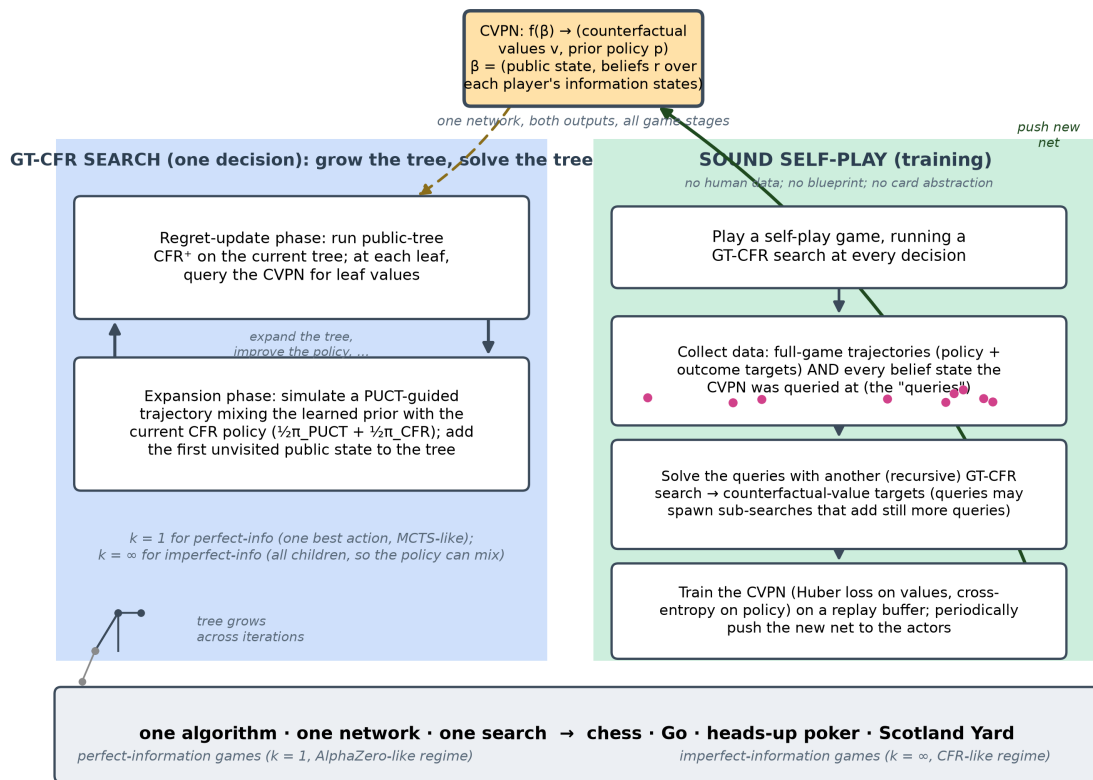
В продължение на седемдесет години двете големи традиции в игровия изкуствен интелект вървяха по отделни пътища. Едната — минимакс, алфа–бета, търсене с дърво на Монте Карло и в крайна сметка AlphaZero — завладя игрите с *пълна информация*, като комбинира търсенето с научена функция на стойността, а най-общото ѝ изражение, AlphaZero, овладя шах, шог и Го с един алгоритъм. Другата — минимизиране на контрафактичното съжаление и четирите покер системи в тази глава — завладя покера с *непълна информация* чрез игровитеоретично разсъждение. Но пътищата никога не се срещнаха. AlphaZero не можеше да играе покер: неговото търсене с дърво на Монте Карло е *некоректно* при скрита информация, по причината, която разделът за Recursive Belief-based Learning изясни — стойността на едно състояние зависи от вероятността, с която се достига до него, така че стойностите не могат просто да бъдат върнати назад от локални поддървета. А покер системите не можеха да играят Го: те бяха изградени около структурата на залаганията на една игра. Дори Recursive Belief-based Learning, най-общата от тях, беше представена и гарантирана само за непълна информация, *свеждайки се* до AlphaZero в случая с пълна информация, вместо наистина да обхваща и двата класа.

Отвореният въпрос, на който Student of Games отговаря, е дали *един* алгоритъм — една процедура за търсене, една архитектура на мрежата, един цикъл на самообучение — може да бъде **коректен и силен и в двата класа игри едновременно**. Трудността не е просто инженерна. Търсенето при пълна информация цели да разшири най-добрата линия дълбоко и да прочете една единствена стойност от всеки **възел**; търсенето при непълна информация трябва да поддържа *разпределение* върху действията (за да не изтича частна информация) и да разсъждава за *убеждение* относно скритите състояния във всеки **възел**. Съгласуването на тези два подхода в един алгоритъм означава намиране на търсене, което разширява дърво по начина, по който го прави AlphaZero — насочено, асиметрично, по всяко време — като същевременно изчислява *игровитеоретично коректните* величини, които CFR изчислява. Student of Games затваря тази празнина, като изгражда точно това търсене, GT-CFR, върху представянето на публичното убеждение, което Recursive Belief-based Learning беше установило, и като обучава своите мрежи чрез *правилно самообучение*, чиито цели произтичат от самото търсене. Резултатът е кулминацията на главата: точката, в която дъгите абстракция-към-невронна и извън линия-към-търсене са съединени от трета — пълна и непълна информация, обединени.

6.6.2 Архитектура

Student of Games е, по своята структура, **цикълът на самообучение на AlphaZero, при който търсенето с дърво на Монте Карло е заменено от GT-CFR и при който мрежата стойност/политика е дефинирана върху публични убеждения** — офлайн фаза, която обучава мрежа чрез самообучение, и онлайн фаза, която търси с нея, но при която същото търсене вече се изпълнява и в двете фази (Фигура 6.5).

Представянето е наследено от линията ReBeL: **общодостъпно състояние на вярванията** \$ pairs the public state (in poker, the betting history and board) with a **range** \$ — двойка разпределения върху информационните състояния, които всеки играч може тайно да заема. Игрите с пълна информация са просто дегенеративният случай, при който всяко публично състояние има точно *едно* информационно състояние и убеждението е точкова маса, което е именно причината една и съща репрезентация да може да обслужва и двата класа.



Фигура 5: Student of Games: the GT-CFR search loop (left) and the sound self-play training loop that feeds it (right) share one CVPN; the footer strip names the unification across perfect- and imperfect-information games.

Върху тази репрезентация се намират двата основни компонента. **Контрафактичната мрежа за стойност и политика (CVPN)** е една единствена мрежа, която при подадено състояние на убеждението излъчва както вектор от контрафактични стойности (по една за всяко информационно състояние, за всеки играч), така и предходна стратегия — една мрежа, която изпълнява и двете задачи за всеки етап от играта, докато DeepStack използва отделни мрежи само за стойност на рунд. **GT-CFR търсенето** е механизмът, който превръща тази мрежа в стратегия, а цикълът на **правилно самообучение** е този, който обучава мрежата въз основа на собствения изход от търсенето. Офлайн актьорите играят игри чрез самообучение, изпълняват GT-CFR търсене при всяко решение и генерират два вида тренировъчни данни — пълни игрови траектории и състоянията на убеждението, при които мрежата е била запитана по време на търсене — докато треньорите настройват нова CVPN и периодически я връщат обратно. Онлайн агентът изпълнява същото GT-CFR търсене (чрез продължително пререшаване), за да избере всеки ход. Класическите градивни елементи са всички видими — CFR⁺ (Стъпка 3) е вътрешният цикъл на търсенето, приближение на стойност и политика (Стъпка 5) е CVPN, а публични убеждения и разлагане (линията DeepStack/ReBeL) са репрезентацията — но свързващата новост е GT-CFR, търсенето, което *разраства* своето дърво, и правилното самообучение, което поддържа всяко търсене съгласувано с всяко друго.

6.6.3 Ключова иновация: Growing-Tree CFR и правилно самообучение

Приносът на Student of Games е търсещ алгоритъм, който за игри с непълна информация прави това, което търсенето с дърво Монте-Карло направи за игри с пълна информация — интелигентно разраства дърво към линиите, които имат значение — *без* да жертва коректността, която изисква скритата информация. Две идеи го правят работещ, а една единствена теорема го прави общ.

Първата идея е **разрастване на дървото чрез редуване на две фази**. MCTS на AlphaZero разширява един възел при всяка симулация и никога не се връща назад, защото в игра с пълна информация стойността на даден възел, след като бъде изчислена, вече не се променя — бъдещето не влияе върху миналото. При непълна информация това не е вярно: както отбелязва Шмид, наблюдението на *бъдещо* действие на противника променя вашето *убеждение* за частното състояние, което той е държал в миналото, така че всяка промяна на стратегията в дървото се разпространява навсякъде. Затова едно търсене не може просто да разширява и оценява; след всяко разширяване то трябва да преизчислява *цялото* дърво, за да го поддържа самосъгласуван. GT-CFR прави именно това. Всяка итерация изпълнява последователно две фази. **Фазата на актуализация на съжалението** извършва няколко итерации на Public-Tree CFR+ върху *текущото* дърво и където то спре — на своите гранични листа — запитва CVPN за контрафактичните стойности на подиграта отдолу, точно както решаването с ограничена дълбочина използва научена стойност на хоризонта. **Фазата на разширяване** след това разраства дървото: тя симулира една единствена траектория от корена, избирайки действия по rUCT правило, което *смесва* предварителната стратегия на мрежата с текущата cFR policy, и добавя първото публично състояние, до което достига и което все още не е в дървото. Итерирането — „разшири дървото, подобри стратегията, разшири, подобри“ — дава търсене *по всяко време*, което, подобно на MCTS, насочва изчисленията към релевантните линии, но за разлика от него решава за игрално-теоретично съгласувана стратегия на всяка стъпка.

Втората идея е **единственият параметър, който адаптира това едно търсене към двата класа игри**. Кога-

то разширява, AlphaZero добавя само едното най-обещаващо действие, което е идеално, когато оптималната игра може да бъде детерминистична. Но оптималната **игра с непълна информация** обикновено е *стохастична* — трябва да се смесват действия, за да се избегне изтичане на информация — така че ангажирането само с едно действие е грешно. GT-CFR следователно разширява първите k действия по предварително убеждение и задава $k = 1$ за игри с пълна информация (една добра линия е достатъчна; търсенето тогава на практика е това на AlphaZero) и $k = \infty$ — всички деца — за **игри с непълна информация** (така че стратегията може да смесва действия между тях). С $k = \infty$ търсенето дори получава *крайно-времева* гаранция за качеството на стратегията, а не само такава в крайна сметка. Този един превключвател е техническото сърце на уеднаквяването: същият код се свежда до търсене, подобно на MCTS, върху шахматна позиция и до итерация, подобна на CFR, върху покер решение.

Третият елемент е **правилното самообучение**, което обучава CVPN. Подобно на AlphaZero, агентът се обучава да предсказва резултата от собственото си търсене: всяко запитване към мрежата, направено по време на търсене, дефинира под-игра, тази под-игра се пререшава чрез *друго* GT-CFR търсене и мрежата се обучава към изхода от търсенето (с по-високо качество) — стойности чрез Huber loss, стратегия чрез кръстосана ентропия, като резултатите от цялата игра осигуряват останалите целеви стойности. Думата *правилно* е носещата: всяко търсене, използвано за генериране на данни, трябва да бъде съгласувано с мрежата и с другите търсения по траекторията (то не трябва да сглобява фрагменти от две различни равновесия), което се осигурява чрез изпълнение на търсенията върху спомагателна игра за безопасно пререшаване. Това е аналогът при непълна информация на самообучението с бутстрапинг на AlphaZero и именно това кара научените стойности действително да сходяват към игровитеоретичните.

Двойката идва с гаранция, която завършва семейството от **граници на коректност** в тази глава. Теорема 1 гласи, че съжалението на средната стратегия на GT-CFR след T итерации се разделя на член, който акумулира грешката ϵ на мрежата за стойности върху *границата* на дървото (листата, които CVPN оценява) и обикновен член на съжалението на CFR върху *вътрешността* на дървото (възлите, където CFR работи); разделяйки на T , експлоатируемостта на средната стратегия е ограничена от

$$\text{exploitability}(\bar{\pi}^T) \lesssim \underbrace{|\mathcal{F}| \epsilon}_{\text{value-net error (frontier)}} + \underbrace{\frac{|\mathcal{N}| UA}{\sqrt{T}}}_{\text{CFR convergence (interior)}},$$

с $|\mathcal{F}|$ и $|\mathcal{N}|$ съответно размерите на границата и вътрешността, U най-голямата разлика в стойностите и A максималният брой действия. Формата вече е позната — член за грешка в стойността плюс $1/\sqrt{T}$ термин за сходимост на CFR, пряк структурен наследник на този при deepStack's $k_1\epsilon + k_2/\sqrt{T}$ и ReBeL's $\delta C_1 + \delta C_2/\sqrt{T}$ — но две неща са отличителни. Първо, коефициентите са *размерите на растящата граница и вътрешност на дървото*, така че границата е буквално твърдение за инкрементален растеж на дървото: добавянето на възли с течение на времето не струва нищо по отношение на реда на сходимост. Второ, и решаващо, **това същото твърдение важи както за дърво на игра с пълна информация, така и за публично дърво с непълна информация** — това е единствената гаранция за коректност, която обхваща и двата класа. Съпътстващ резултат, Теорема 2, след това показва, че извикването на GT-CFR рекурсивно чрез продължително пререшаване в рамките на цял епизод поддържа агента коректен, като експлоатируемостта нараства

само *линейно* с дължината на играта (фактор приблизително $5D + 2$ за D стъпки на пререшаване) — свойството, което позволява на Student of Games да оцелее в хоризонта от двадесет и четири рунда на Scotland Yard. Заедно теоремите казват това, което AlphaZero можеше само да предполага: с подобряването на мрежата (*andsearchdeepens()*) и задълбочаването на търсенето ($\square \text{MATH110} \square$), играта сходи към равновесие на Наш — сега както за покер, така и за шах.

6.6.4 Ограничения, задънени улици и какво остава неразгледано в статията

В съответствие с модела на главата, основният текст в *Science Advances* представя чистата, високо-ниво история, а инженерната откровеност е изложена в приложението, като теоремите са представени там „само неформално“, докато архитектурите, хиперпараметрите, пълният псевдокод, изчислителната мощност и доказателствата са прехвърлени към Допълнителния текст.

Основното ограничение е открито признато: **Student of Games е по-слаб от AlphaZero в шах и Го при еднакви ресурси**, а разликата не е малка в Го — най-силната му конфигурация спечели само *2 от 400* игри срещу AlphaZero в средна фаза на обучение, т.е. загуби 99,5%, дори докато смазва класическата програма Pachi с над 1100 ело точки. Авторите наричат това „цената на общността на SoG“ и формулират причината точно: CFR просто е по-малко ефективен от търсенето в дърво на Монте Карло при игри с пълна информация, така че унифицираното търсене плаща за своята широчина с пикова сила. Шмид откровено заявява, че това е по дизайн — целта е един алгоритъм „по-добър от хората“ във всичко, а не най-добрият шахматен двигател — но това означава, че унификацията е доказателство за *коректност и компетентност* в различните класове, а не за доминация в нито един от тях.

Две стени за мащабиране са признати като реалните ограничения. Първата е **експлозията в пространството на убежденията**, която Шмид определя като основното ограничение: CVPN трябва да *изброи информационните състояния за всяко публично състояние*, така че неговата цена се мащабира с броя на частните състояния, които играч може да заеме — управляемо за 1326 раздавания в покера, но „забранително скъпо в някои игри“, където представянето „се разпада“. Това е точно ограничението за PBS-вход на ReBeL, наследено непокътнато; статията предлага да се направи приближаване с генеративен модел, който *избира проби* от състояния на света вместо да ги изброява, но не изгражда такъв. Втората е **изискването за известен модел**: подобно на AlphaZero и ReBeL, Student of Games се нуждае от перфектен симулатор на правилата, което според думите на Шмид го поставя „зад линията на AlphaZero“ спрямо muZero, който *научава* своя модел.

Няколко по-малки детайла допълват картината. Системата все още използва **рандомизирана абстракция на залозите** в покера (около двадесет хиляди действия, сведени до четири или пет) — единственото експертно познание, което не е премахнато и е посочено като бъдеща работа. Нейното търсене е **квадратично спрямо броя на итерациите** ($O(kT^2)$) извиквания на мрежата, тъй като дървото нараства с всяка стъпка) — неефективност, която авторите отбелязват, че може да бъде премахната чрез паралелизация, но не е била. Хиперпараметърът c — колко актуализации на съжалението да се изпълняват при всяко разширяване — е признато **недостатъчно проучен** („ $c = 1$ не винаги е най-добрият в практиката ... надяваме се да го проучим по-задълбочено“). А чистият аргумент за сходимост на обучението е **идеализиран**, валиден само „асимптотично, когато $T \rightarrow \infty$ и при много голяма (експоненциална) памет“, така че практиката разчи-

та на крайния капацитет на мрежата. Нито един от тези недостатъци не подкопава резултата, но заедно те определят Student of Games като първо, умишлено общо доказателство за концепция, а не като настроен и мащабируем продукт.

6.6.5 Изчислителна мощност и достъпност

Разходите на Student of Games са концентрирани **извън линия, в обучение чрез самообучение с мащаб TPU**, и статията ги отчита *спрямо AlphaZero, а не в абсолютни стойности* — умишлен избор, който позволява честно сравнение без претенции за хардуерен рекорд. Базовата линия на AlphaZero използва 3500 едновременно изпълнители, всеки на един Google TPUv4, в продължение на 800 000 стъпки на обучение, а Student of Games „е обучен с подобно количество изчислителна мощност TPU“, като обучението по домейни варира от приблизително един милион стъпки (Го) до седемнадесет милиона (Scotland Yard). Самото пояснение на Шмид е, че шахът и покерът могат да бъдат обучени „на няколко GPU“, докато Го „безспорно най-трудното“ е изисквало голям пул от TPU — главно защото хардуерът е бил наличен, а не защото методът го е изискувал. Важно е да се отбележи, че **няма една единствена сума в долари** от рода на тази, с която Pluribus стана известен: тъй като сравнението е обвързано с *бюджет за мрежови извиквания*, а не с реално време, честният извод е „сравним с AlphaZero и значителен“ — всъщност авторите включват намаляването на тази изчислителна мощност сред своите нерешени проблеми. По време на игра търсенето е по всяко време и може да се настройва, но неговият $O(kT^2)$ растеж (който се свежда до $O(T)$ за игри с пълна информация, където възелът трябва да бъде оценен само веднъж) означава, че силните конфигурации също не са евтини.

По отношение на достъпността присъдата е смесена и заслужава да бъде изразена точно. **Пълният агент Student of Games и неговите обучени мрежи не бяха публикувани** — няма декларация за достъпност на кода освен данните в статията и допълнението — така че по оста „главен артефакт“ той е заедно със затворения Libratus и Pluribus, а не с отворения ReBeL. Това, което е отворено, е заобикалящата екосистема: същата група DeepMind/Alberta поддържа **OpenSpiel**, който предоставя cFR family и самите еталонни тестове (ледюк, лъжливи зарове, Scotland Yard), така че компонентите и средите са възпроизводими, въпреки че сглобената система не е. На практика това прави Student of Games *отворен метод върху отворена основа със затворен флагман* — разумна средна позиция, но такава, която оставя пълното възпроизвеждане на резултатите за шах/Го/покер/Scotland Yard извън обсега на типична група.

6.6.6 Силни страни и ограничения

Силата на Student of Games е в **обобщеност с коректност** — комбинация, която нито една предишна система в тази глава не предлагаше. Това е първият алгоритъм, който е *доказуемо коректен както за игри с пълна, така и за игри с непълна информация* (Теорема 1–2), и той подкрепя теорията с широчина, която никой конкурент не можа да достигне: в един-единствен дизайн достига експертно до професионално ниво в **шах и Го, побеждава Slumbot**, най-силния публично достъпен бот за хедс-ъп безлимит покер (с около +7 mbb/g — единицата за коефициент на печалба от предишните раздели, която тази статия записва като mbb/hand — и решаващо е, че *не* бива експлоатиран от сондаж за най-добър локален отговор, който хваща по-слабите ботове, които той побеждава с +434 mbb/g), и **побеждава PimBot**, агента за Scotland Yard на най-съвременното ниво, изграден върху търсене в дърво на Монте Карло с ръчно настроени евристики, дори когато на PimBot

са дадени десет милиона симулации за търсене спрямо няколкостотин на Student of Games. Той използва **без човешки данни, без предварително изчислен план и без абстракция на картите**, като научава всичко от самообучение; той **се мащабира** с изчислителна мощност (повече търсене и повече обучение понижават експлоатируемостта, емпиричното лице на Теорема 1); и демонстрира, че *усъвършенства своята политика по време на тестване чрез търсене*, „за разлика от всеки чист алгоритъм за обучение с подкрепление“ — най-ясното изявление в главата за търсене като извод.

Ограниченията са точната форма на неговата амбиция. Той е **по-слаб от специалистите** в техните собствени области — особено забележимо в Го — признатата цена за един алгоритъм за всичко. Неговите гаранции, подобно на тези на Recursive Belief-based Learning, важат само за **игра за двама с нулева сума** (детективите в Scotland Yard са обединени в един отбор, за да се вместят), така че *многопотребителската* празнина в сигурността, разкрита от Pluribus, остава недокосната дори тук. Неговото **представяне на убеждения не се мащабира** до игри с огромни частни състояния и то **изисква известен модел** на средата — двете стени, които самите автори отбелязват първи. То запазва малка **абстракция на залозите**, неговото търсене е **квадратично** по отношение на итерациите и — уникално неудобно за твърдението за обобщеност — **флагманският код никога не беше публикуван**. Честният прочит е, че Student of Games доказва, че уеднаквяването е *възможно и коректно*, и плаща цената на това доказателство в пикова сила, мащаб и възпроизводимост.

6.6.7 Наследство и съвременна значимост

Student of Games е завършекът на аргумента в тази глава и обединява трите ѝ сюжетни линии едновременно. В рамките на петте системи полето премина от **ръчно изградена абстракция към научено невронно приближение** (DeepStack и ReBeL премахнаха абстракцията, от която зависеха Libratus и Pluribus), и от **решаване на всичко офлайн към обучение и търсене** (планът се сви от петабайтите на Libratus до мрежа за стойности и решаване в реално време). Student of Games добавя третото и последно уеднаквяване — **пълна и непълна информация в един алгоритъм** — и с това преформулира цялата сюжетна линия. Концептуалният извод е, че рецептата на AlphaZero (самообучение, научена мрежа за стойности и политики и дърво, пораснало чрез насочено търсене) никога не е била специфична за пълна информация; тя просто се нуждаеше от *коректно* търсене върху *убеждения*, а не върху състояния. Заменете **търсене в дърво на Монте Карло** с GT-CFR и го изпълнете върху публични убеждения, и същата рецепта обхваща шах и покер с една единствена гаранция за сходимост. Това е централната теза на главата, изразена буквално: един алгоритъм, една мрежа, едно търсене, четири много различни игри.

Отношенията му към околния фронт са изясняващи. Student of Games е *общата* рамка, към която ReBeL насочи — той развива своето дърво вместо да фиксира ограничена по дълбочина под-игра, отделя търсенето по време на изпитване от обучението (така че, подобно на AlphaZero, може просто да търси по-усилено по време на изпитване за по-силна игра) и е валидиран в различни класове игри, а не само върху игри с непълна информация. Най-видимият му *непокрит* съсед е **MuZero**: докато MuZero премахва нуждата на AlphaZero от известен модел чрез научаване на динамиката, Student of Games все още предполага перфектен симулатор, така че естествената следваща стъпка — обединяване на GT-CFR с научен модел и генеративна извадка на убеждения — би разтворило и двете му посочени ограничения наведнъж. И той се намира на една ос от **CICERO**, играча на Diplomacy на Meta, който достига човешко ниво, като пренася търсене плюс

обучение в игра със седем играчи, със смесени мотиви и на естествен език: ярко напомняне, че Student of Games уеднаквява *информационната структура*, но не и *броя играчи*. Няколко от неговите части вече са използвани сами по себе си — **GT-CFR** като търсене с нарастващо дърво, полицейно насочено и валидно за всеки клас игри; **CVPN** като единна глава за стойности и политики върху състояния на убеждения; **правилно самообучение** като начин за буутстрапинг на целеви стойности от взаимно съгласувани под-търсения; и **независимо от домейна продължително пререшаване**, което вече не се опира на специалната структура на покера.

За тази дисертация по-специално Student of Games е едновременно таван и пътепоказател. Като най-общата **рамка за звуково търсене**, базирана на PBS, тя е естественият контекст за **принос 1**: нейното убеждение — разпределение върху възможните информационни състояния на противника — е точно субстратът, който трябва да бъде *обогатен* с убеждения относно *типа* стратегия на противника, превръщайки убеждение, търсещо Неш равновесие, в адаптивно такова. И неговото внимателно ограничаване на всички гаранции до **двуйгрови игра с нулева сума** — статията изрично посочва, че Неш е „по-малко значим“ извън нея — преформулира, от най-напредналата гледна точка в литературата, точния пропуск, към който е насочен **принос 2**: уеднаквяването на *информационната структура* на игрите не направи нищо, за да затвори *многоагентната безопасност*, която Pluribus първо разкри. Това е най-чистият възможен преход от тази глава към останалата част от дисертацията, а **синтезът**, който следва, обединява петте системи — **таблица на еволюционните промени** за това какво всяка добави и загуби, диаграмата на еволюцията и картата за повторна употреба на компоненти — за да направи нерешените проблеми и приносите, които ги адресират, явни.

6.7 Синтез

С петте системи на разположение синтезът ги обединява — не за да определи победител, а за да открие три основни момента: формата на седемгодишната еволюция, общите механизми, които системите използват, и нерешените проблеми, с които тази глава предава щафетата на останалата част от дисертацията. Оценъчните карти за всяка система вече съдържат подробностите по отделните измерения, така че тук се прави обобщение, а не повторно извеждане на тези данни.

6.7.1 Дъгата в едно четене

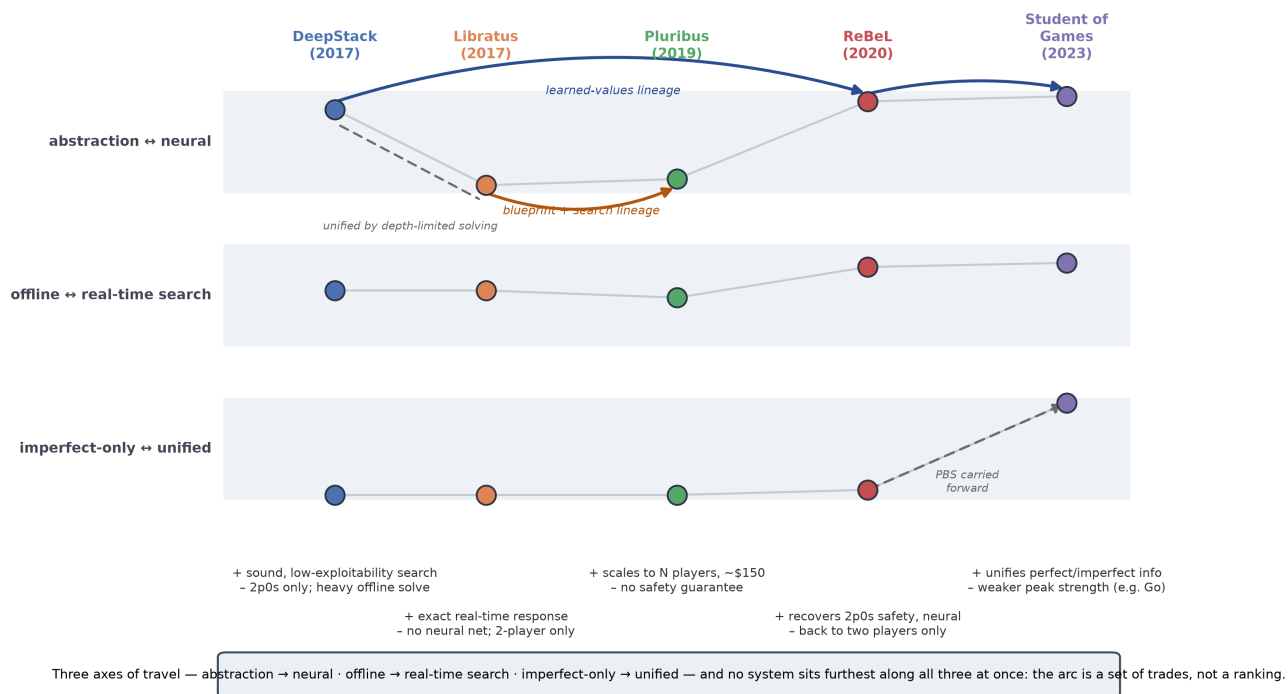
Прогресията е най-добре да се разглежда не като изкачване, а като разходка по трите оси от въведението, при която всяка стъпка напред по една ос беше платена с нещо друго. DeepStack и Libratus откриха десетилетието, като атакуваха една и съща слабост — парадигмата „абстракция и план“ със загуби — от противоположни посоки: DeepStack изостави парадигмата и я замени с научени стойности и продължително пререшаване, докато Libratus я запази и добави доказуемо безопасна корекция в реално време. И двете системи останаха двуйгрови и нито една от тях не съобщи за резултата на другата, така че още в началото „по-добър“ вече беше многоизмерен. Pluribus след това придвижи оста *брой играчи* до шест — първия свръхчовешки резултат във всеки еталонен тест отвъд двама играчи или отбори — но го направи, като се отказа напълно от безопасността и остана табличен — странично движение по представителната ос и назад по гаранциите. ReBeL обърна и двете тези отстъпки, връщайки се към двама играчи, за да *възстанови* гаранция за коректност, като същевременно придвижи представителната ос най-далеч досега — публични състояния на убеждения със

стойности и политики, обучени чрез самоигра, без абстракция и без план — a Student of Games завърши картината, като добави третата ос, обединявайки игра с пълна и непълна информация в един алгоритъм, за сметка на пиковата сила във всяка от тях. Честният едноредов обобщен извод е, че областта замени *ръчно изградена широчина с научена общност и предварително изчисление с търсене в реално време*, и че нито една система не доминира по всички оси наведнъж.

Таблицата по-долу представя тази разходка като отчет за това какво е добавила всяка система и каква е била цената, което представлява полезното сравнение при еволюционен прочит; фиксираните стойности по измерения (играчи, тип игра, план, невронна компонента, търсене, абстракция, пълна информация, изчислителна мощност, ключова иновация) са във всяка оценъчна карта на раздел и тук не се повтарят.

Система (година)	Какво добавя	Какво жертва / каква е цената
DeepStack (2017)	Първо <i>коректно</i> търсене с ограничена дълбочина при скрита информация; научени контрафактични стойности вместо съхранена стратегия; изоставя парадигмата „абстракция и план“ на цялата игра	Все още запазва рядка абстракция на залозите в рамките на своето предвиждане; никога не е тестван директно срещу предишни ботове; приблизително 175 CPU-години офлайн етикетиране на стойности
Libratus (2017/18)	Първа директна <i>и</i> човешка победа чрез вложено безопасно решаване на под-игри в реално време с доказуема граница; самоусъвършенстващ се за една нощ, който поправя собствените си пропуски	Запазва и зависи от абстракция + петабайтов план; без неврално обобщаване; изчислителна мощност в мащаба на суперкомпютър (~25 милиона процесорни часа)
Pluribus (2019)	Първа свръхчовешка многоиграща (шест играчи) игра; прочуто евтина (~150 долара на един сървър)	Отказва се от <i>всички</i> гаранции за безопасност (без N-игрови граници); разчита на <i>небезопасно</i> търсене; остава изцяло табличен и абстрахиран

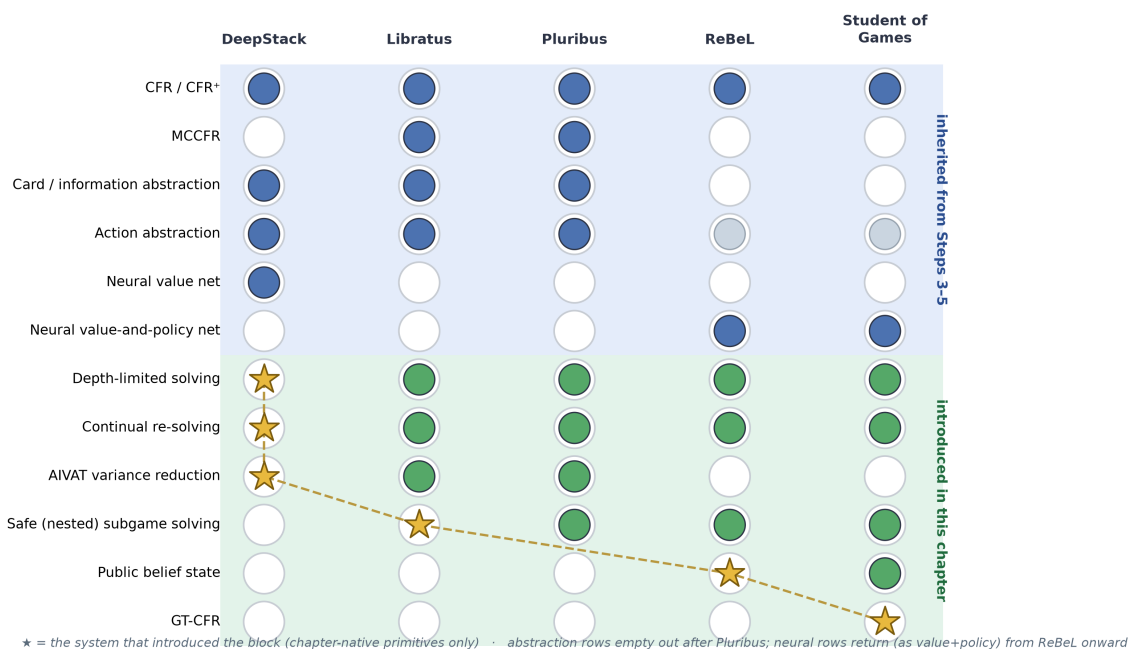
Система (година)	Какво добавя	Какво жертва / каква е цената
ReBeL (2020)	„AlphaZero за непълна информация“: публични състояния на убежденията + научена стойност/политика + CFR в пространството на убежденията; <i>възстановява</i> гаранцията за 2p0s; елиминира абстракцията <i>и</i> плана; с отворен код за Liar’s Dice	Гаранциите се връщат към двама играчи; експлозия в пространството на убежденията там, където общото знание е оскъдно; обучение върху GPU клъстер; изисква известен модел
Student of Games (2023)	Обединява пълна и непълна информация в един коректен алгоритъм (GT-CFR + една мрежа за стойност и политика + правилно самообучение)	По-слаб от специалистите (осезаемо при Го); гаранциите все още са 2p0s; експлозия в пространството на убежденията; изисква известен модел; основният код не е публикуван



Фигура 6: The seven-year arc as trade-offs: the five systems placed along three axes (abstraction → neural; offline → real-time search; imperfect-only → unified), with lineage arrows and a per-system capability-gained/given-up tag.

6.7.2 Какво се пренася напред

Почти нищо в тези системи не е изцяло ново; всяка от тях представлява рекомбинация на малък набор от градивни елементи, повечето от които са въведени в стъпки 3–5. **CFR и CFR⁺** (стъпка 3) са вътрешният решаващ на всяка система; **Монте Карло CFR** (стъпка 3) обучава планове на Libratus и Pluribus; **абстракция на действията** (стъпка 4) е в основата на Libratus и Pluribus и оцелява само като остатъчно меню за залагания в ReBeL и Student of Games; а **невронно приближение на стойността** — а след това на **стойност и стратегия** (стъпка 5) е именно това, което позволява на deepStack, ReBeL и Student of Games да разтворят абстракцията, от която зависят останалите. Върху тези глават налагат свои собствени примитиви, всеки въведен от една система и използван повторно от нейните наследници: **ограничено по дълбочина решаване, продължително пререшаване, безопасно вложено решаване на под-игри, публично състояние на убежденията и Growing-Tree CFR**. Прочетете надолу списъка с *първи появи* и еволюцията на главата се преразказва сама чрез компоненти: блоковете се натрупват, тези от ерата на абстракцията избледняват, а научените поемат контрол.



Фигура 7: Component-reuse map: Step 3–5 building blocks and chapter-native primitives (rows) against the five systems in chronological order (columns); stars mark the system that introduced each native primitive, connected by a staircase of first appearances.

6.7.3 Защо това е важно за нашето изследване

Тази глава представлява съвременното равнище, върху което дисертацията се изгражда и от което се отклонява, а нейният най-важен принос към тази работа е рамкиращ. Всяка система, разгледана тук, е **независима от противника по дизайн**: всяка изчислява най-лошосценарийна устойчива стратегия и я играе безусловно, а двете многопотребителски и тествани с хора системи правят тази позиция явна — те отказват да моделират или да се адаптират към противниците си, за да не бъдат контраексплоатирани, а Pluribus дори не знае

срещу кого играе. Прочетена като обзор, следователно, главата е каталог на пет системи от световна класа, които споделят точно една пропуск: нито една не се адаптира към конкретния, погрешим противник пред себе си. Тази обща пропуск представлява отвора, който заема цялата тази дисертация, а останалата част от този подраздел разглежда активите на главата спрямо трите планирани приноса.

За принос 1 (Рамка за поведенческа адаптация). Най-прекият принос на главата е **публичното състояние на убеждението (PBS)**, въведено от ReBeL и наследено от Student of Games — разпределение върху скритите състояния, които противникът може да заема при наличие на общо знание, което принос 1 разширява, за да носи убеждения относно *типа стратегия* на противника, а не само неговите карти. Два по-малко очевидни момента изострят това. Първо, PBS е изобретен, за да направи *търсенето доказуемо коректно*, а коректността е ортогонална на моделирането на противника — така че изграждането на адаптация върху PBS наследява целия преизчислителен механизъм и неговите гаранции за сходимост *непроменени*, позволявайки на адаптивното убеждение да бъде актуализирано вътре в доказуемо коректно търсене, вместо да бъде добавено отвън; адаптацията получава възможността да използва повторно не само структура от данни, но и доказателствата, прикрепени към нея. Второ, всяка система в главата споделя **двустепенна архитектура** — евтина глобална стратегия (план или научена мрежа за стойности), усъвършенствана чрез локално решаване в реално време — и собственият дизайн на дисертацията, „започнете от безопасна стратегия и се подобрявайте срещу слаби противници“, е същата двустепенна форма, при която целта на второто ниво е променена от *точност* към *ограничена експлоатация*. Следователно главата предоставя както субстрата за адаптация, така и системния скелет, в който тя се вписва.

За принос 2 (Многоагентна безопасна експлоатация). Ключовият елемент е Pluribus: свръхчовешки покер за шестима играчи, постигнат *без* каквато и да е гаранция за безопасност, което е точно пропускът, който една теория за многоагентна безопасна експлоатация трябва да запълни — точка, която главата заявява директно. По-фин прочит превръща собствената уговорка на системите в оправдание за приноса. Pluribus и Student of Games и двете ограничават своите гаранции до игра за двама с нулева сума и признават, че извън нея равновесието на Наш нито е уникално, нито може да бъде изчислено ефективно, нито дори представлява гаранция срещу загуба. Ако равновесието не осигурява безопасност в многопотребителската среда от самото начало, тогава отказът да се експлоатира видимо слаб противник означава отказ от гаранция, която никога не е била притежавана — което преформулира умишлената експлоатация от безразсъдно отклонение от безопасността в *рационална цел*, след като безопасността е недостижима само чрез равновесие. Това е най-чистата мотивация, която литературата предлага за многоагентната експлоатация, която принос 2 преследва.

За принос 3 (Методология за оценяване). Глава стандартизира **експлоатируемостта** — най-лошата загуба спрямо **най-добре реагиращ противник** — като универсален измерител, прилаган еднакво към всичките пет системи, и тази однородност сама по себе си представлява шаблон за **рамка за оценяване между системи**. Тя също така предоставя на дисертацията два инструмента, предназначени за нова употреба. **Локалната най-добра реакция (LBR) сонда**, използвана в цялата глава за *сертифициране* на **неексплоатируемост**, се превръща в *тест за безопасност*, когато бъде насочена към **адаптивен агент** — прави ли преследването на слаб противник вас **контраексплоатируем**? — което е точно антагонистичната, стил „учебна атака“ оценка, от която се нуждае стъпка 8. А AIVAT, **намалителят на дисперсията на научената стойност** зад докладваните резултати на DeepStack и Pluribus, е инструментът, който прави малкото, но *растящо* предимство на

адаптацията статистически видимо срещу брутлната дисперсия на картите игри, където **суровите печалби** биха подвели.

По-нататъшни опорни точки (кандидат — за запазване или премахване според крайната дължина). Следните са компактни екстраполации, а не твърдения, вече направени от системите; всяко представлява едноредово **начално число** за по-късни стъпки.

- **Многократно използвана форма на доказателство (Принос 2).** Всяка гаранция в главата има една и съща форма — термин за приближение на стойността плюс $1/\sqrt{T}$ термин за сходимост (DeepStack's $k_1\epsilon + k_2/\sqrt{T}$, Libratus's 2Δ , ReBeL's $\delta C_1 + \delta C_2/\sqrt{T}$, SoG's $|\mathcal{F}|\epsilon + |\mathcal{N}|UA/\sqrt{T}$) — което дава на Принос 2 готов шаблон за ограничаване на експлоатируемостта, *породена* от умишлено отклонение.
- **Експлоатация с ограничена дълбочина (Принос 1/Принос 2).** Многозначните листа на Pluribus (k избираеми стратегии за продължение) структурно представляват меню от поведения на противника на хоризонта; преинтерпретирането на това меню като *типове* на противника води до управляема експлоатация отвъд дълбочината на търсенето — точният проблем, който стъпка 8 „адаптиране отвъд границата на дълбочината“ разглежда.
- **Карта за изграждане спрямо цитиране (методология).** Подразделите „Изчислителна мощ и достъпност“ сортират областта на това, което е възпроизводимо в мащаба на докторска дисертация (Single-Server CFR на Pluribus, CPU решавач с един поток на ReBeL, отворената OpenSpiel среда, малки игри) спрямо това, което е само за цитиране (свръхкомпютърът на Libratus, TPU басейнът на SoG) — неясната обосновка за изграждане върху евтината, отворена линия (задачата reBeL-Lite-on-Leduc), вместо възпроизвеждане на водещ продукт.
- **Пренасочен механизъм (рамкиране).** Самоусвършенстването през нощта на Libratus е корекция на стратегията в мача, насочена към *собствените* му слабости; дисертацията изпълнява същия цикъл наблюдение—последвано от корекция, насочен към *противника*, така че приносът е пренасочване на доказан свръхчовешки механизъм, а не неговото изобретяване.
- **Кука за разказ за границата (рамкиране).** И петте системи вече са цитирани като ранни инстанции на „изчислителна мощ по време на тест“ — изразходване на търсене в момента на вземане на решение, вместо всичко да бъде вградено в предварително изчислена стратегия — така че рамкирането на адаптацията в реално време като изчислителна мощ за *адаптация* по време на тест свързва тази работа със съвременния разказ за моделите на разсъждение.
- **Съгласуваност като безопасност (Принос 2).** „Правилното самообучение“ на Student of Games изисква всяко локално търсене да остане в *съответствие* с безопасна референция, концептуално същото ограничение като регулираната с КЛ-дивергенция (PiKL) експлоатация, която Принос 2 ще използва — аналогия, която си струва да бъде заета.
- **Абстракция, прехвърлена към типово пространство (Принос 3).** Урокът от стъпка 4, че абстракцията със загуби създава експлоатируеми шевове, се пренася от пространството на състоянията към пространството на *типовете* на противника: твърде грубият типови модел е експлоатируем по същия начин, както твърде грубата абстракция на карти, предсказвайки режим на отказ за Принос 3 да улови.

6.7.4 Нерешени проблеми и предаването към стъпки 7–15

Четири нерешени проблема произтичат пряко от синтеза и всеки е начално число за по-късна стъпка. Първият е самата **невидимост за противника**: цялата граница изчислява фиксирана, **стратегия, оптимална в най-лошия случай** и я играе безусловно, оставяйки празнината между статично равновесие и динамичен, **съобразен с противника начин на игра** широко отворена — предмет на стъпка 7 (извеждане на поведението на противника от **следи от действия**) и извеждащата част на **принос 1**. Вторият е **безопасност в мултиплейър среда**: Pluribus доказва, че методите „**търсене по Наш и търсене**“ *печелят* в N-играч игри с **непълна информация**, без да предлагат никаква **гаранция за безопасност**, а Student of Games потвърди, че дори най-общата **рамка за звуково търсене** все още ограничава своите гаранции до двама играчи — така че управляема теория на *безопасна експлоатация отвъд игра за двама с нулева сума* (стъпка 8, **принос 2**) остава наистина нерешена. Третият е **адаптация отвъд границата на дълбочината**: решаващите в реално време системи в главата гледат само малко напред и носенето на **модел на противника** по звуков начин отвъд този хоризонт — вместо да се изхвърля на листа — е нерешен проблем, който стъпка 8 разглежда пряко. Четвъртият е **бюджети за изчисления в реално време**: разходите на системите варират в шест порядъка и **адаптивен агент**, който трябва да пререшава и да преоценява противник онлайн, наследява най-строгата версия на това ограничение.

Тези проблеми определят фаза D (стъпки 7–8), критичното ядро на дисертацията, и се разклоняват през следващите фази — динамика на многоагентни системи и формиране на коалиции (фаза E), поведенчески конвейери, задвижвани от данни, които основават моделите върху реални следи (фаза F), и рамката за междудоменна оценяване, която затваря плана (фаза G). Петте системи в тази глава колективно представляват платформата, върху която стъпва настоящата дисертация, както и контрапункта, спрямо който тя се определя: тя наследява техните представяния на състояния на убежденията, тяхното търсене в реално време и тяхната дисциплина по отношение на експлоатируемостта и се стреми да добави една способност, която всички те, по дизайн, пропускат — способността да забележи, че противникът не играе оптимално и да предприеме действия по въпроса, без самата тя да стане експлоатируема в замяна.